

""Here is my writeups.""

Then,let's start.

Reserve

逆向工程入门指北 (345y! ! !)

用csdn上大佬分享的新版ida pro，打开下载的chall1

看到main函数部分

```
strcpy(s2, "moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}");
```

结束。

flag:

moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}

或许会有其他方法，但我这种应该挺简单

```
sunlight_tr4vel3r:sunlight_traveler ~/逆向工程入门指北 — in MoeCTF 2026 [12:32:31]
$ moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}
[*] 正在提交: moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/逆向工程入门指北 — in MoeCTF 2026 [12:32:40]
$
```

Assembly(汇编爱好者狂喜)

直接用记事本打开下载的chall2.txt

看向第一行，很显然，看到mov ecx,25，立刻记下：ecx寄存器目前储存的值：25

还有lea rsi,[byte 404000]，那我们看到404000：

byte_404000:

db 0x6d, 0x6f, 0x65, 0x63, 0x74, 0x66, 0x7b

db 0x41, 0x73, 0x73, 0x65, 0x6d, 0x62, 0x31, 0x79

db 0x5f, 0x4c, 0x34, 0x6e, 0x67, 0x75, 0x61, 0x67, 0x65, 0x5f

熟知16进制的朋友们都知道，0x开头是16进制的标志。

直接解ascii码，得到

往下看，看到loop段，这里在复制[rsi]到[r di]，再分别+1,记得我们之前记下的ecx=25吗，这一段每loop一次，ecx-1知道0结束。那么我们从404000按顺序抽25个字节。（正好25个）

按顺序解一下ascii码：moectf{Assemb1y_L4nguage_，显然是flag前半段了。

继续往下看

eax = 0x2a

再加0x16

再cmp比较与0x40的值。

如果你不能直接看出来，那么就算：0x2a转10进制= $2*16+10=42$

$0x16=16+6(10)=22(10)$

$42+22=64$

$0x40=4*16=64=eax$

对于jne指令，此时cmp的结果是zf=1,, 不跳转。

接着往下看

ebx=0x10=16

shl ebx,2, 相当于 $ebx*4=64$

与eax相等，同理不跳。

接着往下看

ecx=0x39

ecx-=0x20

$ecx=0x19>0x18$

对于jg指令，此时cmp的结果是zf=0且sf=of, 跳转。

那么我们查看401050的相关内容。

ecx=15, 先记下

往下看401060的内容，我们立刻知道，401050-401060这段是从404020获取15个字节，每个分别与0x42异或。然后解ascii码。

计算：

原字节	异或 0x42	字符
0x73	0x31	1
0x11	0x53	s
0x1d	0x5f	_
0x21	0x63	c
0x2d	0x6f	o
0x72	0x30	0
0x2d	0x6f	o
0x72	0x30	0
0x0d	0x4f	o
0x2d	0x6f	o
0x2d	0x6f	o
0x2e	0x6c	l
0x63	0x21	!
0x63	0x21	!
0x3f	0x7d	}

1S_co0o0Oool!!!}很显然是flag右半边了。

那么结束

flag: moectf{Assembly_L4nguage_1S_co0o0Oool!!!}

```

sunlight_tr4vel3r:sunlight_traveler ~/Assembly — in MoeCTF 2026 [12:34:27]
$ moectf{Assembly_L4nguage_1S_co0o0Oool!!!}
[*] 正在提交: moectf{Assembly_L4nguage_1S_co0o0Oool!!!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Assembly — in MoeCTF 2026 [12:34:31]
$ _

```

bbxor

老样子, ida打开, 来看main函数, 直接伪代码启动。

牢玩家肯定很快就看懂了: 实际上这里的意思是: 程序要求输入36个10进制整数, 存入v4, 并与对应的 cipher 值比对。当全部满足时, 输出yes

```

printf("Input: ");
for ( i = 0; i <= 35; ++i )
    __isoc99_scanf("%d", &v4[i]);

for ( j = 0; j <= 35; ++j )

```

```

{
    if ( (v4[j] ^ 0x66) != cipher[j] )
    {
        printf("no");
        return 0;
    }
}
printf("yes");
return 0;

```

我们直接双击这个cipher，看看跳转到哪

```

.data:0000000000404040 cipher      dd 0Bh, 9, 3, 5, 12h, 0, 1Dh, 24h, 15h, 0Fh, 5, 39h
.data:0000000000404074          dd 1Eh, 56h, 14h, 39h, 5, 0Eh, 7, 57h, 0Ah, 3, 8, 1, 3
.data:00000000004040A8          dd 39h, 15h, 9, 0Ah, 10h, 3, 2, 2 dup(47h), 1Bh

```

熟知16进制的牢玩家们肯定知道，h后缀是16进制的标志。

那么数据我们全得到了，不想手算，就写python脚本。

```

cipher = [
    0x0B, 0x09, 0x03, 0x05, 0x12, 0x00, 0x1D, 0x24,
    0x52, 0x15, 0x0F, 0x05, 0x39, 0x1E, 0x56, 0x14,
    0x39, 0x05, 0x0E, 0x07, 0x57, 0x0A, 0x03, 0x08,
    0x01, 0x03, 0x39, 0x15, 0x09, 0x0A, 0x10, 0x03,
    0x02, 0x47, 0x47, 0x1B
]

flag = ''.join(chr(c ^ 0x66) for c in cipher)
print(flag)

##### ##输出程序实际需要输入的十进制整数

nums = [c ^ 0x66 for c in cipher]
print(' '.join(map(str, nums)))

```

结果：moectf{B4sic_x0r_cha1lenge_solved!!}

```

109 111 101 99 116 102 123 66 52 115 105 99 95 120 48 114 95 99 104 97 49 108 101 110 103 101
95 115 111 108 118 101 100 33 33 125

```

答案已经显然，我们还是把下面这些数带入input检验，果然是yes

flag: moectf{B4sic_x0r_cha1lenge_solved!!}

```

sunlight_tr4vel3r:sunlight_traveler ~/bbxor _____ in MoeCTF 2026 [12:35:11]
$ moectf{B4sic_x0r_challenge_solved!!}
[*] 正在提交: moectf{B4sic_x0r_challenge_solved!!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/bbxor _____ in MoeCTF 2026 [12:56:25]
$

```

反方向的RC4

你懂的，ida打开，查看main函数，伪代码启动。

```

int __fastcall main(int argc, const char **argv, const char **envp)

```

```

{
    _BYTE v4[32]; // [rsp+0h] [rbp-40h] BYREF
    char s[32]; // [rsp+20h] [rbp-20h] BYREF

    memset(s, 0, 0x15uLL);
    memset(v4, 0, 0x14uLL);
    printf("Input something: ");
    fgets(s, 21, _bss_start);
    init_cipher(v4);
    rc4_crypt(v4, 20LL, "Th1s_1s_secret~!", 16LL);
    puts("Done.");
    return 0;
}

```

看不出来啥，我们必须查看rc4_crypt以及init_cipher，去看看吧

rc4_crypt函数:

```

__int64 __fastcall rc4_crypt(__int64 a1, int a2, __int64 a3, int a4)
{
    __int64 result; // rax
    _BYTE v5[256]; // [rsp+18h] [rbp-120h]
    char v6; // [rsp+121h] [rbp-17h]
    char v7; // [rsp+122h] [rbp-16h]
    char v8; // [rsp+123h] [rbp-15h]
    unsigned int k; // [rsp+124h] [rbp-14h]
    int v10; // [rsp+128h] [rbp-10h]
    int j; // [rsp+12Ch] [rbp-Ch]
    int v12; // [rsp+130h] [rbp-8h]
    int i; // [rsp+134h] [rbp-4h]

    for ( i = 0; i <= 255; ++i )
        v5[i] = i;
    v12 = 0;
    for ( j = 0; j <= 255; ++j )
    {
        v12 = (unsigned __int8)(v5[j] + v12 + *(_BYTE *)(j % a4 + a3));
        v6 = v5[j];
        v5[j] = v5[v12];
        v5[v12] = v6;
    }
    v10 = 0;
    v12 = 0;
    for ( k = 0; ; ++k )
    {
        result = k;
        if ( (int)k >= a2 )
            break;
        v10 = (unsigned __int8)(v10 + 1);
        v12 = (unsigned __int8)(v5[v10] + v12);
        v8 = v5[v10];
        v5[v10] = v5[v12];
        v5[v12] = v8;
        v7 = v5[(unsigned __int8)(v5[v10] + v5[v12])];
        *(_BYTE *)((int)k + a1) ^= v7;
    }
    return result;
}

```

```
}
```

反正是个加密流程，实在不行直接复制粘贴给python(当然，语法需要调整)。现在，我们需要找密钥与密文。

双击一下这个Th1S_1s_secret~!，跳转到

```
.rodata:0000000000402010      public g_key
.rodata:0000000000402021      db 'Th1S_1s_secret~!',0 ; DATA XREF: main+79f0
.rodata:0000000000402021      align 10h
.rodata:0000000000402030      public g_cipher
.rodata:0000000000402030      db 0x2h ; DATA XREF: init_cipher+27f0
.rodata:0000000000402031      db 24h ; $
.rodata:0000000000402032      db 24h ; $
.rodata:0000000000402033      db 3
.rodata:0000000000402034      db 0x3h
.rodata:0000000000402035      db 0x4h
.rodata:0000000000402036      db 41h ; A
.rodata:0000000000402037      db 62h ; b
.rodata:0000000000402038      db 11h
.rodata:0000000000402039      db 0xf3h
.rodata:000000000040203A      db 8Ah
.rodata:000000000040203B      db 28h ; (
.rodata:000000000040203C      db 0A0h
.rodata:000000000040203D      db 71h ; q
.rodata:000000000040203E      db 9Bh
.rodata:000000000040203F      db 0xEh
.rodata:0000000000402040      db 27h ; '
.rodata:0000000000402041      db 46h ; F
.rodata:0000000000402042      db 19h
.rodata:0000000000402043      db 0A9h
.rodata:0000000000402044      ; const char format[]
.rodata:0000000000402044      format      db 'Input something: ',0 ; DATA XREF: main+38f0
.rodata:0000000000402056      ; const char s[]
.rodata:0000000000402056      s      db 'Done.',0 ; DATA XREF: main+8Df0
.rodata:0000000000402056      _rodata      ends
```

高手都看出来了，g_key指的是密钥，g_cipher指的是密文。

这个g_cipher值可以直接复制粘贴了：b2 24 24 03 e3 b4 41 62 11 f3 8a 28 a0 71 9b be 27 46 19 a9

g_key也只要解一下ascii码：54 68 31 53 5f 31 73 5f 73 65 63 72 65 74 7e 21

那么我们着手编写python代码，或者上网抄一份，数值自己填。

```
def rc4_crypt(data: bytes, key: bytes) -> bytes:
```

```
    S = list(range(256))
```

```
    j = 0
```

```
    for i in range(256):
```

```
        j = (j + S[i] + key[i % len(key)]) & 0xFF
```

```
        S[i], S[j] = S[j], S[i]
```

```
    out = bytearray(data)
```

```
    i = 0
```

```
    j = 0
```

```
    for n in range(len(out)):
```

```
        i = (i + 1) & 0xFF
```

```
        j = (j + S[i]) & 0xFF
```

```
        S[i], S[j] = S[j], S[i]
```

```
        out[n] ^= S[(S[i] + S[j]) & 0xFF]
```

```
    return bytes(out)
```

```
key = bytes.fromhex("546831535f31735f7365637265747e21")
```

```
cipher = bytes.fromhex("b2242403e3b4416211f38a28a0719bbe274619a9")
```

```
plain = rc4_crypt(cipher, key)
```

```
print("ascii:", plain.decode('latin-1'))
```

运行即可。

flag: moectf{OH~Dyn4mic!}

```
sunlight_tr4vel3r:sunlight_traveler ~/反方向的 RC4 ————— in MoeCTF 2026 [13:00:34]
$ moectf{OH~Dyn4mic!}
[*] 正在提交: moectf{OH~Dyn4mic!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/反方向的 RC4 ————— in MoeCTF 2026 [13:19:10]
```

Ultra Potato Xplosion

这次先读题目吧，显然提示我们是UPX加壳。那么牢办法暂时被ban了，我们需要先脱壳。

这里用wsl2.

```
upx -d chall8 -o unpacked
```

再用ida打开unpacked，老办法。

这下一眼可见Fnrr68qUFDSTA5tupNAp8pgrFPJNesF3R6KNk

并且从input flag可知，我们在得到flag后可以来比对。正确就会输出correct，否则no

熟知base64的高手们知道，这个不可能是base64（长度不符，也没=）

那么可能是base58吗？相当有可能（无0、O、I、l、=）

解一下试试看

Fnrr68qUFDSTA5tupNAp8pgrFPJNesF3R6KNk

输入格式 Base58

输出格式 String

字符集

UTF-8

Base58 编码

Base58 解码

复制

下载

清空

转换结果

moectf{EaSy_UPX_anD_Base58}

那么结束

flag: moectf{EaSy_UPX_anD_Base58}

```
sunlight_tr4vel3r:sunlight_traveler ~/Ultra Potato Xpl... ————— in MoeCTF 2026 [13:19:22]
$ moectf{EaSy_UPX_anD_Base58}
[*] 正在提交: moectf{EaSy_UPX_anD_Base58}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Ultra Potato Xpl... ————— in MoeCTF 2026 [13:37:47]
```

奇怪的APP

再见了ida小弟，这个你打不开。

首先我们需要一个能分析.apk的工具

这里我使用jadx

```
package com.moectr.apkreverse;

import android.app.Activity;
import android.os.Bundle;

/* JADX INFO: loaded from: classes.dex */
public class MainActivity extends Activity {
    public static String 00o00o0() {
        return "RXYzcnNlXzFz";
    }

    @Override // android.app.Activity
    protected void onCreate(Bundle bundle) {
        super.onCreate(bundle);
    }
}
```

在MainActivity中，这个RXYzcnNlXzFz实在显眼，正好12长度，试试base64解码

Base64 编码/解码

将文本编码为 Base64 或将 Base64 解码回文本格式。

RXYzcnNlXzFz

字符编码: UTF-8

☒ 解码过滤非 Base64 字符

Base64 编码

Base64 解码

↕ 交换

清空

Ev3rse_1s

Ev3rse_1s

先放着。继续看下面的内容。

```
{
  "api": "local",
  "debug": false,
  "theme": "warmup",
  "cache_key": "Z2hUPyEhIX0=",
  "note": "Nothing interesting here... probably."
}
```

位于config.json，又是一段Z2hUPyEhIX0=，12长度还有=，base64启动

ghT?!!!}

右括号很爽，确定这个是最右边的内容。

继续往下。

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">APK Reverse Warmup</string>
    <string name="looks_like_a_theme_key">X2Z1Tl9yMQ==</string>
</resources>
```

位于strings.xml

依旧base64

_fuN_r1

是中间内容，不妨先拼一下试试。

Ev3rse_1s_fuN_r1ghT?!!!}

顺便猜一下，这个肯定是?? reverse is fun right

继续往下

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.moectf.apkreverse"
    android:versionCode="1"
    android:versionName="1.0">
    <uses-sdk
        android:minSdkVersion="21"
        android:targetSdkVersion="28"/>
    <uses-permission android:name="bW9LY3Rme0Fwa19S"/>
    <application android:label="APK Reverse Warmup">
        <activity
            android:name="com.moectf.apkreverse.MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

位于AndroidManifest.xml

你懂的，解码去

moectf{Apk_R

好啦！结束。

flag: moectf{Apk_REv3rse_1s_fuN_r1ghT?!!!}

请你喝茶

老办法，你懂的。

这次信息不多，但是我们可以看到有个check_tea_part, 和check_xtea_part

查看相关函数。

在check_tea_part函数伪代码中，双击get_tea_key_address，来到对应函数，再双击tea_key，看到一些16进制值，并且下方看到tea_cipher和相邻的几个地址。不妨把值记下来，可能就是密钥，密文的值。

```
.rodata:0000000000402010 tea_key db 78h ; x ; DATA XREF: get_tea_key_address+810
.rodata:0000000000402011 db 56h ; V
.rodata:0000000000402012 db 34h ; 4
.rodata:0000000000402013 db 12h
.rodata:0000000000402014 | db 21h ; !
.rodata:0000000000402015 db 43h ; C
.rodata:0000000000402016 db 65h ; e
.rodata:0000000000402017 db 87h
.rodata:0000000000402018 db 68h ; h
.rodata:0000000000402019 db 24h ; $
.rodata:000000000040201A db 57h ; W
.rodata:000000000040201B db 13h
.rodata:000000000040201C db 57h ; W
.rodata:000000000040201D db 13h
.rodata:000000000040201E db 68h ; h
.rodata:000000000040201F db 24h ; $
.rodata:0000000000402020 xtea_key db 0EFh ; DATA XREF: get_xtea_key_address+810
.rodata:0000000000402021 db 0BEh
.rodata:0000000000402022 db 0ADh
.rodata:0000000000402023 db 0DEh
.rodata:0000000000402024 db 0BEh
.rodata:0000000000402025 db 0BAh
.rodata:0000000000402026 db 0FEh
.rodata:0000000000402027 db 0CAh
.rodata:0000000000402028 db 44h ; D
.rodata:0000000000402029 db 33h ; 3
.rodata:000000000040202A db 22h ; "
.rodata:000000000040202B db 11h
.rodata:000000000040202C db 88h
.rodata:000000000040202D db 77h ; w
.rodata:000000000040202E db 66h ; f
.rodata:000000000040202F db 55h ; U
.rodata:0000000000402030 tea_cipher dd 0B3E7E33Eh ; DATA XREF: check_tea_part+8E1r
.rodata:0000000000402034 dword_402034 dd 0B4114672h ; DATA XREF: check_tea_part+A21r
.rodata:0000000000402038 dword_402038 dd 8E088C0Ch ; DATA XREF: check_tea_part+B61r
.rodata:000000000040203C dword_40203C dd 14B2C329h ; DATA XREF: check_tea_part+CA1r
.rodata:0000000000402040 xtea_cipher dd 60EC68ABh ; DATA XREF: check_xtea_part+8E1r
.rodata:0000000000402044 dword_402044 dd 940251CEh ; DATA XREF: check_xtea_part+A21r
.rodata:0000000000402048 dword_402048 dd 0B427534Ch ; DATA XREF: check_xtea_part+B61r
.rodata:000000000040204C dword_40204C dd 0DF435416h ; DATA XREF: check_xtea_part+CA1r
.rodata:0000000000402050 aInputYourFlag db 'Input your flag: '.0
```

结合tea6solve，这些确实就是我们要找的值。密文直接抄。在网上简单学习db小端序解析算法，自己算一下，就能获得密钥。那么现在只剩下填解密函数了。

```
cipher1[4] = {

    0xB3E7E33E,

    0xB4114672,

    0x8E088C0C,

    0x14B2C329

};

key1[4] = {

    0x12345678,

    0x87654321,

    0x13572468,
```

```

        0x24681357

};

cipher2[4] = {

    0x60EC68AB,

    0x940251CE,

    0xB427534C,

    0xDF435416

};

key2[4] = {

    0xDEADBEEF,

    0xCAFEFEBABE,

    0x11223344,

    0x55667788

};

```

看tea_encrypt函数

```

__int64 __fastcall tea_encrypt(unsigned int *a1, _DWORD *a2)
{
    __int64 result; // rax
    unsigned int i; // [rsp+20h] [rbp-10h]
    int v4; // [rsp+24h] [rbp-Ch]
    unsigned int v5; // [rsp+28h] [rbp-8h]
    unsigned int v6; // [rsp+2Ch] [rbp-4h]

    v6 = *a1;
    v5 = a1[1];
    v4 = 0;
    for ( i = 0; i <= 0x1F; ++i )
    {
        v4 -= 1640531527;
        v6 += (v5 + v4) ^ (16 * v5 + *a2) ^ ((v5 >> 5) + a2[1]);
        v5 += (v6 + v4) ^ (16 * v6 + a2[2]) ^ ((v6 >> 5) + a2[3]);
    }
    *a1 = v6;
    result = v5;
    a1[1] = v5;
    return result;
}

```

把里面vx, ax代表啥找出来，往里填就行

```

{
    cipher[1] -= (cipher[0] + delta) ^ (16 * cipher[0] + key[2]) ^ ((cipher[0] >> 5) + key[3]);
    cipher[0] -= (cipher[1] + delta) ^ (16 * cipher[1] + key[0]) ^ ((cipher[1] >> 5) + key[1]);
    delta += 1640531527;
}

```

xtea_encrypt也差不多

```

{
    cipher[1] -= (((cipher[0] << 4) ^ (cipher[0] >> 5)) + cipher[0]) ^ (key[(delta >> 11) & 3] + delta);
    delta -= 0x9E3779B9;
    cipher[0] -= (((cipher[1] << 4) ^ (cipher[1] >> 5)) + cipher[1]) ^ (key[delta & 3] + delta);
}

```

那么运行，结束

flag: moectf{Wh4t_a_n1ce_cup_of_TEA!!}

```

sunlight_tr4vel3r:sunlight_traveler ~/请你喝茶 ————— in MoeCTF 2026 [13:57:16]
$ moectf{Wh4t_a_n1ce_cup_of_TEA!!}
[*] 正在提交: moectf{Wh4t_a_n1ce_cup_of_TEA!!}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/请你喝茶 ————— in MoeCTF 2026 [14:21:43]

```

Tell Me No Lies

老办法。

在main函数中，我们看到一些端倪。

```

int __fastcall main(int argc, const char **argv, const char **envp)
{
    char v4[32]; // [rsp+0h] [rbp-50h] BYREF
    _BYTE s[32]; // [rsp+20h] [rbp-30h] BYREF
    unsigned int v6; // [rsp+40h] [rbp-10h]
    unsigned int v7; // [rsp+44h] [rbp-Ch]
    __int64 v8; // [rsp+48h] [rbp-8h]

    memset(s, 0, 0x15uLL);
    memset(v4, 0, sizeof(v4));
    puts("=== MOE Restricted Recovery Console ===");
    puts("[*] <mark>Checking debugger state...</mark>");
    v7 = <mark>IsDebuggerPresent();</mark>
    if ( v7 )
    {
        puts("[!] <mark>Debugger detected.</mark>");
        puts("[-] <mark>Recovery console terminated</mark>.");
        <mark>return 1</mark>;
    }
    else
    {
        puts("[*] Verifying operator token...");
        build_required_token(s);
        <mark> v8 = acquire_operator_token();</mark>
        <mark> v6 = compare_operator_token(v8, s);</mark>
        if ( v6 )
        {
            puts("[!] <mark>Operator token mismatch.</mark>");
            puts("[-] <mark>Recovery console terminated.</mark>");
        }
    }
}

```

```

    <mark>return 1</mark>;
}
else
{
    puts("[*] Authorization accepted.");
    puts("[*] Starting multilayer recovery engine...");
    if ( (unsigned int)self_driven_decrypt(v7, v6, v4) )
    {
        puts("[+] Recovery complete.");
        printf("[+] %s\n", v4);
        memset(v4, 0, sizeof(v4));
        return 0;
    }
    else
    {
        puts("[!] Recovery state corrupted.");
        puts("[-] No valid record recovered.");
        return 1;
    }
}
}
}
}

```

debugger detect, 反调试。

但是再看看题目的信息：

藏起你身后的目光，让我相信此刻无人注视：IsDebuggerPresent()

再让那场注定失败的校验归于通过：compare_operator_token()

我便会把沉睡的秘密亲口告诉你：self_driven_decrypt()

再看看那个return 1

如果强行return 0，会怎么样？

现在我们需要更多信息。

wsl, 启动!

```

(base) sun11ght_tr4ve13r@Sleepymas07:/mnt/d$ file chall10
chall10: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, not
stripped
(base) sun11ght_tr4ve13r@Sleepymas07:/mnt/d$ checksec --file=chall10
RELRO           STACK CANARY      NX            PIE            RPATH
RUNPATH        Symbols      FORTIFY Fortified      Fortifiable    FILE
Partial RELRO   No canary found  NX enabled    No PIE          No RPATH      No
RUNPATH        59 Symbols      No            0              3chall10

```

关键：

- **not stripped**: 符号表完整，函数名一个都没少，IDA 打开直接 F5 就能读；
- **No PIE**: 基址固定 0x400000，所有地址可以写死，下断点、改内存都毫无障碍。

先跑一下试试

```
(base) sun11ght_tr4vel3r@Sleepymas07:/mnt/d$ ./chal110
=== MOE Restricted Recovery Console ===
[*] Checking debugger state...
[*] Verifying operator token...
[!] Operator token mismatch.
[-] Recovery console terminated.
```

那么, gdb动态调试。

现在先在ida中看看它们的地址。

```
.text:0000000000401256 IsDebuggerPresent proc near          ; CODE XREF: main+4C↓p
.text:0000000000401256
.text:0000000000401256 s1              = byte ptr -110h
.text:0000000000401256 var_106          = byte ptr -106h
.text:0000000000401256 stream          = qword ptr -10h
.text:0000000000401256 var_8           = qword ptr -8
.text:0000000000401256
.text:0000000000401256 ; __unwind {
✓.text:0000000000401256          endbr64
```

IsDebuggerPresent:0x401256

```
.text:00000000004014D6
.text:00000000004014D6 ; ===== S U B R O U T I N E =====
.text:00000000004014D6 ; Attributes: bp-based frame
.text:00000000004014D6
.text:00000000004014D6 ; __int64 __fastcall compare_operator_token(_QWORD, _QWORD)
.text:00000000004014D6 compare_operator_token proc near          ; CODE XREF: main+A6↓p
.text:00000000004014D6
.text:00000000004014D6 s2              = qword ptr -10h
.text:00000000004014D6 s1              = qword ptr -8
.text:00000000004014D6
.text:00000000004014D6 ; __unwind {
✓.text:00000000004014D6          endbr64
```

compare_operator_token:4014D6

```
.text:0000000000401E55 loc_401E55:          ; CODE XREF: main+B3↑j
.text:0000000000401E55          mov     edi, offset aAuthorizationA ; "[*] Authorization accepted."
.text:0000000000401E5A          call    _puts
.text:0000000000401E5F          mov     edi, offset aStartingMultil ; "[*] Starting multilayer recovery engine"...
.text:0000000000401E64          call    _puts
.text:0000000000401E69          mov     eax, [rbp+var_10]
.text:0000000000401E6C          mov     ecx, eax
.text:0000000000401E6E          mov     eax, [rbp+var_C]
.text:0000000000401E71          lea     rdx, [rbp+var_50]
.text:0000000000401E75          mov     esi, ecx
.text:0000000000401E77          mov     edi, eax
.text:0000000000401E79          call    self_driven_decrypt
.text:0000000000401E7E          test    eax, eax
```

self_driven_decrypt:401e79

现在写gdb脚本

```
set pagination off
set confirm off

break *0x401256
commands
    silent
    return (int)0
continue
end

break *0x4014d6
commands
    silent
    return (int)0
```

```
    continue
end

break *0x401e7e
commands
    silent
    printf "\n"
    continue
end

run
```

```
(base) sunlight_tr4vel3r@sleepymas07:/mnt/d$ nano wit.gdb
(base) sunlight_tr4vel3r@sleepymas07:/mnt/d$ gdb -nx -q -batch -x wit.gdb
./chall10
Breakpoint 1 at 0x401256
Breakpoint 2 at 0x4014d6
Breakpoint 3 at 0x401e7e

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
=== MOE Restricted Recovery Console ===
[*] Checking debugger state...
[*] Verifying operator token...
[*] Authorization accepted.
[*] Starting multilayer recovery engine...

[+] Recovery complete.
[+] moectf{Wow~h0w_do_Y0u_f1nd_IT?}
[Inferior 1 (process 1944) exited normally]
```

结束。完美。

flag: moectf{Wow~h0w_do_Y0u_f1nd_IT?}

```
sunlight_tr4vel3r:sunlight_traveler ~/Tell Me No Lies — in MoeCTF 2026 [14:22:13]
$ moectf{Wow~h0w_do_Y0u_f1nd_IT?}
[*] 正在提交: moectf{Wow~h0w_do_Y0u_f1nd_IT?}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Tell Me No Lies — in MoeCTF 2026 [15:48:53]
```

成都好室友

#牢办法又没有用了, sad

竟然打不开, 查查成分

```
(base) sunlight_tr4vel3r@sleepymas07:/mnt/d$ xxd -g1 -l 32 chall15.pyc
00000000: 6f 0d 0d 0a 00 00 00 00 62 f5 49 6a 0b 3d 00 00  o.....b.Ij.=..
00000010: e3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
```

```
(base) sunlight_tr4vel3r@Sleepymas07:/mnt/d$ file chall15.pyc
chall15.pyc: Byte-compiled Python module for CPython 3.10 (magic: 3439),
timestamp-based, .py timestamp: Sun Jul 5 06:10:42 2026 UTC, .py size: 15627
bytes
```

文件头没毛病: 6f 0d 0d 0a

版本是python3.10, 先创建虚拟环境

```
conda create -n py310 python=3.10 -y
```

```
conda activate py310
```

然后运行看看

```
(py310) sunlight_tr4vel3r@Sleepymas07:/mnt/d$ python3.10 chall15.pyc
Successfully decode finished: result.png
Image data size: 11116 bytes.
```

看看result.png



观察右下角, 图片被反色!



反色一波, 然后seedream ai补全



扫码得到结果, 结束

flag: moectf{PYC_p1us_QRc0dE_M1ssing_c0rN3r}

```
sunlight_tr4vel3r:sunlight_traveler ~/成都好室友 — in MoeCTF 2026 [16:21:44]
$ moectf{PYC_p1us_QRc0dE_M1ssing_c0rN3r}
[*] 正在提交: moectf{PYC_p1us_QRc0dE_M1ssing_c0rN3r}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/成都好室友 — in MoeCTF 2026 [16:32:09]
$ _
```

让我们说中文

#还是老办法用着爽。

```

__int64 __fastcall main(int a1, char **a2, char **a3)
{
    size_t v3; // rax
    const char *v4; // rax
    char *v5; // rbx
    char v7[4104]; // [rsp+0h] [rbp-1008h] BYREF

    __printf_chk(2LL, "Input cipher: ", a3);
    if ( !fgets(v7, 4096, stdin) )
    {
        puts("Input error.");
        return 1LL;
    }
    v7[strcspn(v7, "\r\n")] = 0;
    v3 = strlen(v7);
    v4 = (const char *)sub_401480(v7, v3);
    v5 = (char *)v4;
    if ( !v4 )
    {
        puts("Memory error.");
        return 1LL;
    }
    if ( !strcmp(
        v4,
        "KwC2gtPmKwn2NwTyKwGiltv0KwOiuqTvKw43utP2KwOxgttPmKwC2gtPmKwn21wUiKwIxnvtzKwn21Ta5Kw42KTP0KwIiNtt0Kw42NRT3KwM2g4"
        "JaKwn3NTarKwIxu5OrKw42NRT3KwOxKwUrKwIxu5OrKw42NRT3KwIxKtP3Kwn3N4PjKwIiK5T0Kw431RUxKwIiNtt0Kw42NRT3Kw42NwOxKwOi"
        "KTPzKwKxg50xKw42g5TpKwDx14JrKwIxu5OrKw42NRT3KwIxKtP3KwM2g4JaKwC2g4P3KwDxKwOaKw42NwMqKwIxu5OrKw42NRT3KwIiN5TzKw"
        "Kxg50xKw42g5TpKwDx14JrKwIiNtt0Kw42NRT3KwOxK4JaKwKxg50tKwGx1wU5Kwn3N4PjKwIiK5T0KwT31TPzKwn3N4PjKwIiK5T0KwKxg5Mq"
        "KwIxu5OrKw42NRT3Kw431vPjKwIxu5OrKw42NRT3KwM2g4JaKwn3NTarKw42KTPqKwn21Ta5Kw42KTP0" ) )
    {
        puts("Correct!");
    }
    else
    {
        puts("Wrong!");
        free(v5);
        return 0LL;
    }
}

```

这串乱码很可疑，拿去base64解一下。按照题目给的信息，分别尝试UTF-8和GBK

结果不看了，没那么简单，再去找找看。

```

.rodata:000000000402013 aInputError db 'Input error.',0 ; DATA XREF: main:loc_4011DEf0
.rodata:000000000402020 ; const char reject[]
.rodata:000000000402020 reject db 0Dh,0Ah,0 ; DATA XREF: main+40f0
.rodata:000000000402023 ; const char aMemoryError[]
.rodata:000000000402023 aMemoryError db 'Memory error.',0 ; DATA XREF: main:loc_4011F1f0
.rodata:000000000402031 ; const char aCorrect[]
.rodata:000000000402031 aCorrect db 'Correct!',0 ; DATA XREF: main:loc_4011D0f0
.rodata:00000000040203A ; const char s[]
.rodata:00000000040203A s db 'Wrong!',0 ; DATA XREF: main+81f0
.rodata:000000000402041 align 20h
.rodata:000000000402060 ; const char s2[]
.rodata:000000000402060 s2 db 'KwC2gtPmKwn2NwTyKwGiltv0KwOiuqTvKw43utP2KwOxgttPmKwC2gtPmKwn21wUiK'
.rodata:000000000402060 ; DATA XREF: main+6Ef0
.rodata:0000000004020A1 db 'wIxnvtzKwn21Ta5Kw42KTP0KwIiNtt0Kw42NRT3KwM2g4JaKwn3NTarKwIxu5OrKw'
.rodata:0000000004020E2 db '42NRT3KwOxKwUrKwIxu5OrKw42NRT3KwIxKtP3Kwn3N4PjKwIiK5T0Kw431RUxKwI'
.rodata:000000000402123 db 'iNtt0Kw42NRT3Kw42NwOxKwOikTPzKwKxg50xKw42g5TpKwDx14JrKwIxu5OrKw42'
.rodata:000000000402164 db 'NRT3KwIxKtP3KwM2g4JaKwC2g4P3KwDxKwOaKw42NwMqKwIxu5OrKw42NRT3KwIiN'
.rodata:0000000004021A5 db '5TzKwKxg50xKw42g5TpKwDx14JrKwIiNtt0Kw42NRT3KwOxK4JaKwKxg50tKwGx1w'
.rodata:0000000004021E6 db 'U5Kwn3N4PjKwIiK5T0KwT31TPzKwn3N4PjKwIiK5T0KwKxg5MqKwIxu5OrKw42NRT'
.rodata:000000000402227 db '3Kw431vPjKwIxu5OrKw42NRT3KwM2g4JaKwn3NTarKw42KTPqKwn21Ta5Kw42KTP0'
.rodata:000000000402268 db 0
.rodata:000000000402269 align 20h
.rodata:000000000402280 aSix5starspjyoun db 'Six5starsPJYouNgLLkEw4Ik1nGABCDHFHMOQRUVWXZbcdefghjmqpvzy0236789+/',0 ; DATA XREF: sub_401480+23f0
.rodata:000000000402280 align 20h
.rodata:0000000004022C1 aAbcdefghijklmn db 'ABCDEFGHijklmnopQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/',0 ; DATA XREF: sub_4012F0+48f0
.rodata:0000000004022E0 ; DATA XREF: sub_401480+1Cf0
.rodata:0000000004022E0 _rodata ends

```

这块内容有意思，我们看到下面两行，ABCD....显然是标准base64，那上面的Six5star...是怎么回事呢？难道是自定义映射表？

现在就动手试试看。

得到

```

RTc4OEIxBRTY4NTk1RTZBQ0E3RThBMzk0RTU5MEI4RThCOEEyRTc4OEIxBRTY4QTIBRTVCN0E2RTY4Q
kFDRTU4RkI3RTVBNEE3RTU4Njk5RTg4OUJFRTY5NkFGRTVCMDhGRTU4Njk5RThCRTIGRTVCMDhGR
TU4Njk5RTVCREI5RTY5NUIwRTVBRDk3RTU5QjICRTVBNEE3RTU4Njk5RTU4NThCRThBRki2RTRCOD
hCRTU4ODkyRTdCQUJGRTVCMdHGRU4Njk5RTVCREI5RTg4OUJFRTY5NkFGRTVCMDhGRTU4Njk5RTh
VCMDhGRTU4Njk5RTVBNDk2RTRCODhCRTU4ODkyRTdCQUJGRTVBNEE3RTU4Njk5RThCRUJFRTRC
ODhERTZCQTIDRTY5NUIwRTVBRDk3RTk5QkI2RTY5NUIwRTVBRDk3RTRCODgzRTVCMDhGRTU4Njk
5RTU5Q0lwRTVCMDhGRTU4Njk5RTg4OUJFRTY5NkFGRTU4RkIzRTY4QkFDRTU4RkI3

```

base64? 试试看

E788B1E68595E6ACA7E8A394E590B8E8B8A2E788B1E68A9AE5B7A6E68BACE58FB7E5A4A7E58699E889BEE696AFE5B08FE58699E8BE9FE5B08FE58699E5BDB9E695B0E5AD97E59B9BE5A4A7E58699E5858BE8AFB6E4B88BE58892E7BABFE5B08FE58699E5BDB9E889BEE789B9E7BE8EE58583E5B08FE58699E5A496E4B88BE58892E7BABFE5A4A7E58699E8BEBEE4B88DE6BA9CE695B0E5AD97E99BB6E695B0E5AD97E4B883E5B08FE58699E59CB0E5B08FE58699E889BEE696AFE58FB3E68BACE58FB7

高手都看出来了，6个6个一组。

E788B1/E68595/E6ACA7/E8A394/E590B8/E8B8A2/E788B1/E68A9A/E5B7A6/E68BAC/E58FB7/E5A4A7/E58699/E889BE/E696AF/E5B08F/E58699/E8BE9F/E5B08F/E58699/E5BDB9/E695B0/E5AD97/E59B9B/E5A4A7/E58699/E5858B/E8AFB6/E4B88B/E58892/E7BABF/E5B08F/E58699/E5BDB9/E889BE/E789B9/E7BE8E/E58583/E5B08F/E58699/E5A496/E4B88B//E58892/E7BABF/E5A4A7/E58699/E8BEBE/E4B88D/E6BA9C/E695B0/E5AD97/E99BB6/E695B0/E5AD97/E4B883/E5B08F/E58699/E59CB0/E5B08F/E58699/E889BE/E696AF/E58FB3/E68BAC/E58FB7

按照题目的提示，解成unicode形式

Unicode和UTF编码转换

输入:

对应字符:
爱慕欧裔吸踢爱抚左括号大写艾斯小写辟小写役数字四大写克诶下划线小写役艾特美元小写外下划线大写达不溜数字零数字七小写地小写艾斯右括号

聪明人知道，已经结束了。

#沙威玛，哦沙威玛~~

flag: moectf{Spe4K_e@\$y_W07ds}

```
sunlight_tr4vel3r:sunlight_traveler ~/让我们说中文 ————— in MoeCTF 2026 [17:05:57]
$ moectf{Spe4K_e@$y_W07ds}
[*] 正在提交: moectf{Spe4K_e@$y_W07ds}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/让我们说中文 ————— in MoeCTF 2026 [17:06:01]
```

Mewtype

#牢办法又区了

先打开来看看，复制粘贴出来 (Ctrl+Shift+End)

```
exit(1 - (lambda myk43: (lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)
(len(myk43) == 54))((lambda: (lambda myk13: lambda vol14: myk13(vol14)(myk13))((lambda myk19: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)((myk19 * myk19 * myk19 - myk19) % 6 == 0))(len(myk43))((lambda myk13: lambda vol14: myk13(vol14)(myk13))((lambda vol0: (lambda myk1: myk1(myk1))((lambda vol2: vol0(lambda a: vol2(vol2)(a))))(lambda myk39: lambda vol40: lambda myk41: lambda vol42: (lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(myk41 >= len(vol40) - 1))((lambda: lambda myk3: lambda vol4: myk3)(lambda: (lambda myk13: lambda vol14: myk13(vol14)(myk13))((lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda vol18: (lambda myk15: lambda
```

```

vol16: myk15((lambda vol10: lambda myk11: lambda vol12: vol10(vol12)(myk11))(vol16))(vol16))
((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6:
vol6)((vol18 * vol18 + vol18) % 2 == 0))(lambda myk5: lambda vol6: vol6))(myk41))(lambda: (lambda
vol22: lambda myk23: lambda vol24: lambda myk25: lambda vol26: (lambda myk7: lambda vol8:
lambda myk9: myk7(vol8)(myk9))((lambda vol20: (lambda myk17: (lambda myk3: lambda vol4:
myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol20 == 0))(myk25))(lambda: (lambda
myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol22 +
myk23 + vol24 * vol24 == vol26))((lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))
((lambda myk21: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5:
lambda vol6: vol6)(myk21 == 1))(myk25))(lambda: (lambda myk17: (lambda myk3: lambda vol4:
myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol22 * myk23 + vol24 == vol26))(lambda:
(lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6:
vol6)(vol22 * vol22 + myk23 * myk23 + vol24 * vol24 * vol24 == vol26))))((ord(vol40[myk41]))
(ord(vol40[myk41 + 1]))(ord(vol40[(myk41 + 17) % len(vol40)])))(vol42)((10111, 11290, 137705,
13834, 12017, 882822, 2861, 6952, 148686, 9290, 9962, 153274, 13069, 9595, 871218, 9834,
10851, 1584472, 9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566, 13114, 6109,
140607, 13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923, 10278, 5369,
1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119)[myk41]
^ 90))(lambda: (lambda vol22: lambda myk23: lambda vol24: lambda myk25: lambda vol26:
(lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda vol20: (lambda myk17:
(lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol20 == 0))
(myk25))(lambda: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5:
lambda vol6: vol6)(vol22 + myk23 + vol24 * vol24 == vol26))((lambda myk7: lambda vol8: lambda
myk9: myk7(vol8)(myk9))((lambda myk21: (lambda myk17: (lambda myk3: lambda vol4: myk3) if
myk17 else lambda myk5: lambda vol6: vol6)(myk21 == 1))(myk25))(lambda: (lambda myk17:
(lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol22 * myk23
+ vol24 == vol26))(lambda: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else
lambda myk5: lambda vol6: vol6)(vol22 * vol22 + myk23 * myk23 + vol24 * vol24 * vol24 ==
vol26))))((ord(vol40[myk41 + 1]))(ord(vol40[myk41]))(ord(vol40[(myk41 + 17) % len(vol40)])))(vol42)
((10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686, 9290, 9962, 153274, 13069,
9595, 871218, 9834, 10851, 1584472, 9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921,
152566, 13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923,
10278, 5369, 1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085, 880083, 10095,
15119)[myk41] ^ 90)))(myk39(vol40)(myk41 + 1)((vol42 + 1) % 3)))(myk43)(0)(0))(lambda myk13:
lambda vol14: myk13(vol14)(myk13))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17
else lambda myk5: lambda vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2:
vol0(lambda a: vol2(vol2)(a))))(lambda myk27: lambda vol28: lambda myk29: 0 if myk29 ==
len(vol28) else ord(vol28[myk29]) + myk27(vol28)(myk29 + 1))(myk43)(0) == 5138))((lambda myk13:
lambda vol14: myk13(vol14)(myk13))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17
else lambda myk5: lambda vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2:
vol0(lambda a: vol2(vol2)(a))))(lambda vol30: lambda myk31: lambda vol32: 0 if vol32 ==
len(myk31) else ord(myk31[vol32]) ^ vol30(myk31)(vol32 + 1))(myk43)(0) == 26))((lambda myk13:
lambda vol14: myk13(vol14)(myk13))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17
else lambda myk5: lambda vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2:
vol0(lambda a: vol2(vol2)(a))))(lambda myk33: lambda vol34: lambda myk35: 0 if myk35 ==
len(vol34) else ord(vol34[myk35]) * ord(vol34[myk35]) + myk33(vol34)(myk35 + 1))(myk43)(0) ==
520600))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda
vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2: vol0(lambda a: vol2(vol2)(a))))
(lambda vol36: lambda myk37: lambda vol38: 0 if vol38 == len(myk37) - 1 else ord(myk37[vol38]) *
ord(myk37[vol38 + 1]) + vol36(myk37)(vol38 + 1))(myk43)(0) == 467644)))))))(lambda: lambda myk5:
lambda vol6: vol6)))(input())(1)(0))

```

先别晕，有信息：10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686, 9290, 9962, 153274, 13069, 9595, 871218, 9834, 10851, 1584472, 9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566, 13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923, 10278, 5369, 1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119

查一下，经典的lambda混淆

[Lambda-revenge CTF逆向深度解析：从邱奇编码到约束求解的终极指南-CSDN博客](#)感谢csdn社区的支持。

lambda cond: lambda then: *lambda* else: cond(then)(else)这个相当于if

(lambda myk3: lambda vol4: myk3)这个是True

(lambda myk5: lambda vol6: vol6)这个是False

眼尖的都看到了：

len(myk43) == 54

即字符串输入长度（flag长度）应为54。

继续分析，看一下外层。

exit(1-(lambda myk43:.....)(input()))(1)(0))

查阅资料得知：这个exit(1-p)，如果p=1，就正常退出，否则报错退出。

那么我们来拆解p: (lambda myk43:.....)(input()))(1)(0))

我们的input()值先传给了myk43。

然后查阅资料知道后面的1，0不影响逻辑，那就放着。

这边高亮了一些数值计算。

有异或计算，与26比对。

经过分析（已经瞎了），找到了三种计算分支，由一个数值%3为0，1，2开始分支。

翻译一下。

```
ord(vol40[myk41])           # 第 i 个字符
ord(vol40[myk41 + 1])       # 第 i+1 个字符
ord(vol40[(myk41 + 17) % len(vol40)]) # 第 (i+17) % 54 个字符
```

这段是循环，执行53次

```
def check(s):
    if len(s) != 54:
        return False
    arr = [10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686,
           9290, 9962, 153274, 13069, 9595, 871218, 9834, 10851, 1584472,
           9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566,
           13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540,
           15852, 4864, 1389923, 10278, 5369, 1586542, 10519, 5995, 1281695,
           4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119]
    for i in range(53):
        a = ord(s[i])
        b = ord(s[i+1])
        c = ord(s[(i+17) % 54])
```

```

    target = arr[i] ^ 90
    if i % 3 == 0:
        if a + b + c*c != target:
            return False
    elif i % 3 == 1:
        if a * b + c != target:
            return False
    else:
        if a*a + b*b + c*c*c != target:
            return False
    if sum(ord(ch) for ch in s) != 5138:
        return False
    xor_sum = 0
    for ch in s:
        xor_sum ^= ord(ch)
    if xor_sum != 26:
        return False
    if sum(ord(ch)**2 for ch in s) != 520600:
        return False
    if sum(ord(s[i]) * ord(s[i+1]) for i in range(53)) != 467644:
        return False
    return True

```

这就是这段代码的逻辑。

我们只需要逆向这一段。

写个python脚本吧。

用z3

```

from z3 import *
arr = [
    10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686,
    9290, 9962, 153274, 13069, 9595, 871218, 9834, 10851, 1584472,
    9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566,
    13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540,
    15852, 4864, 1389923, 10278, 5369, 1586542, 10519, 5995, 1281695,
    4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119
]
s = [Int(f's{i}') for i in range(54)]
sol = Solver()
for x in s:
    sol.add(x >= 32, x <= 126)
for i in range(53):
    a, b = s[i], s[i + 1]
    c = s[(i + 17) % 54]
    target = arr[i] ^ 90
    if i % 3 == 0:
        sol.add(a + b + c * c == target)
    elif i % 3 == 1:
        sol.add(a * b + c == target)
    else:
        sol.add(a * a + b * b + c * c * c == target)
sol.add(Sum(s) == 5138)
sol.add(Sum([x * x for x in s]) == 520600)
sol.add(Sum([s[i] * s[i + 1] for i in range(53)]) == 467644)
m = sol.model()

```

```
flag = ''.join(chr(m[x].as_long()) for x in s)
print(flag)
```

运行，完工。

flag:

moectf{l@mbd4_c@lcu1us_4r3_h4rd_but_z3_s0lv3r_1s_3asy}

```
sunlight_tr4vel3r:sunlight_traveler ~/Mewtype ----- in MoeCTF 2026 [17:10:36]
$ moectf{l@mbd4_c@lcu1us_4r3_h4rd_but_z3_s0lv3r_1s_3asy}
[*] 正在提交: moectf{l@mbd4_c@lcu1us_4r3_h4rd_but_z3_s0lv3r_1s_3asy}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Mewtype ----- in MoeCTF 2026 [18:27:51]
$
```

#实际做了两个小时这玩意，回来写wp又看了半小时，晕了

Ultato Polosion Xptra

#终于做出来了。

那么好，ida不能用，题目提示upx，查查成分

```
sunlight_tr4vel3r@Sleepymas07 /m/d> file chall9.3
chall9.3: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), BuildID[sha1]=598c2ed12bc0b902b910c3d21208c26ec39011a3, statically linked, no section header
```

看到一个elf头。

再查查strings

```
sunlight_tr4vel3r@sleepymas07 /m/d> strings ./chall9.3
PJYoung
llikes
walking
/!OH@!
3ao
==@@7
/DoS
; P7
@LoL#
/lib64
nux-x86-
so.2
gUaB
fgets
rlen
art_ma%r
6[LIBC_2.
;349gmon_kt
PTE1
H=8@@
AR!H
D7T6
2BH9
D7T6
Input f
lag:
wrong!
```

GHint
DA pseudocode, __ROL8
n) is a built-1 helpe
r macI
g 64-bi
tate-left;_Fo&ex
amp1S0xFEDCBA987654321
CDirect!
9*3\$"g1't
YT_QRL
RPhupX
ST]H)
t/Hc
]YH9
^U\M1
AP_j
/dev/shm
=>
PX!u
[A\JW
CDvD
<+.u
^A_*
)US1
L48 w
hP@uuw
=z8Br
Z{[r
@bQs
\Z-j
La]f
`A+M
wu%I
xxuV
P0+A
p CN
SRVW
PyK
&>Zo
IuDS
s2V^
/proc/self/ex
7-it
/dev
PM@aW
-5.0 wants memfd_create(), or
need
/(,O_TMPFILE,)
\$Info: This file is packed with the UPX executable packer <http://upx.sf.net> \$
\$Id: UPX 5.20 Copyright (C) 1996-2026 the UPX Team. All Rights Reserved. \$
GCC: (Ubuntu 13.
0-6u
2~24
vy J
aJXl
@GH
YB#X
yyw`

WSWSX
WWSS
ysww
g vZ
}hw&4
k2g+
3;sv3'
4'k5-
H5"@
d8g=
\$~ek/
g'[ly
_off_too
IO_read_ptr
simk
short ,s
o_u64/
Dfileno
Mlo^
ng c
lags1
`uIcoIumn
decv
markerwk
gnedm6w
csp'*cry
0save
/?FILEPuU|
5GNU R
R[s7T -mj
p=x86-6
#f0-
noct
us-Ps
d-0b
aA0!Zkm+
Pyl<
upEkI
m{h!)
x9.c
}f2026
wr/6b/gc
(bis
`"/tDe(.
=Hano
@2mcc2
(@/5F
' .7x=C
_exd
f-\$'@C/
cfpd
gQbk
#5omp#)d
!%ne
`fray_
_)t*
23456
OPFRAME_END

```
GLOBAL_OFFSET_TABL
(Vc_Hr
@&IBC_2
&v.,
gmon
dsH,Q
pi+gss
MC-6
trh
no.m%\4-.c
a(      zt
hghdr
f3p1
?d!d
e0?0A
\$oj
UPX!
UPX!
```

末尾处两个upx，中间有input flag，wrong，CDrect，信息很多而且很乱。

来个具体一点的。

```
sunlight_tr4vel3r@Sleepymas07 /m/d> strings -t x chall9.3 | grep -i 'upx\\|Info'
e13 RPhupX
1946 $Info: This file is packed with the UPX executable packer http://upx.sf.net $
1995 $Id: UPX 5.20 Copyright (C) 1996-2026 the UPX Team. All Rights Reserved. $
270b UPX!
2714 UPX!
```

可以看出是超级魔改

继续深入

```

sunlight_tr4vel3r@Sleepymas07 /m/d> readelf -h chall9.3
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  EXEC (Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x405de0
  Start of program headers:              64 (bytes into file)
  Start of section headers:              0 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              5
  Size of section headers:               64 (bytes)
  Number of section headers:              0
  Section header string table index:      0
sunlight_tr4vel3r@Sleepymas07 /m/d> readelf -l chall9.3

Elf file type is EXEC (Executable file)
Entry point 0x405de0
There are 5 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz             MemSiz              Flags  Align
LOAD             0x0000000000000000 0x0000000000040000 0x0000000000040000
                 0x00000000000001000 0x0000000000040500  RW    0x1000
LOAD             0x0000000000000000 0x0000000000040500 0x0000000000040500
                 0x000000000000019e1 0x000000000004019e1  R E   0x1000
NOTE             0x00000000000000158 0x00000000000400158 0x00000000000400158
                 0x0000000000000030 0x0000000000000030  R     0x8
NOTE             0x00000000000000188 0x00000000000400188 0x00000000000400188
                 0x0000000000000044 0x0000000000000044  R     0x4
GNU_STACK        0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000  RW    0x10

```

结合一下之前的strings

偏移	字符串	信息
0x1946	\$Info: This file is packed with the UPX executable packer ...	是 UPX 壳 (UPX 自己写的 info 串)
0x1995	\$Id: UPX 5.20 Copyright (C) 1996-2026 ...	版本号/版权年份都很新 → 魔改/自制 UPX
0x270B 0x2714	UPX! ×2	尾部有两个 magic: 一个诱饵、一个真 PackHeader
0xBA9 0xBB4 0xBBA	Input f / tag: / wrong!	这题在交互式读 flag 并比较, 攻击面就是比较逻辑
0xC59	CDirect!	成功串 Correct! 的碎片 (o、r 被"吃掉"了)
0xBCE 0xBEB	DA pseudocode, __ROL8 / n) is a built-in helpe	出题人留的提示: 算法里有 __ROL8、IDA 伪代码里的内置函数
0xC30	amp1S0xFEDCBA987654321	提示了一个常量 0xFEDCBA987654321
0x389 0x39B 0x3C0	fgets / rlen / 6[LIBC_2.	.dynstr 碎片 → 导入函数集合 (fgets/strlen/...) → 反推 main 结构
0x2ED 0x2F7 0x305	/lib64 / nux-x86- / so.2	原程序 .interp 的碎片
0x1A42	GCC: (Ubuntu 13.	编译环境 Ubuntu 24.04 + gcc 13
0x23A4 0x23C7	OPFRAME_END / GLOBAL_OFFSET_TABL	原程序 symtab 符号名的碎片 (没 strip)
0x1B8 0x1CF 0x1F5	PJYoung / likes / walking	出题人水印

提前准备一下解析elf头和程序头的python脚本

```
#info.py
import struct, re, sys

fn = sys.argv[1] if len(sys.argv) > 1 else 'chall9.3'
d = open(fn, 'rb').read()
print('len', len(d))

print('--- packer signatures ---')
for pat in [b'UPX', b'UPX0', b'UPX1', b'UPX!', b'$$$ ', b'NRV', b'MPRESS',
b'ASPack', b'FSG',
b'.text', b'.data', b'.shstrtab']:
    idxs = [m.start() for m in re.finditer(re.escape(pat), d)]
```

```

print(' %-10s %-42s %d' % (pat.decode(), (' '.join('%#x' % i for i in
idxs[:8]) or '-'), len(idxs)))

eh = struct.unpack_from('<16sHHIQQQIHMMMM', d, 0)
e_entry, e_phoff, e_shoff, e_phentsize, e_phnum, e_shentsize, e_shnum,
e_shstrndx = \
    eh[4], eh[5], eh[6], eh[9], eh[10], eh[11], eh[12], eh[13]

print('--- ELF header ---')
for n, v in [('e_ident', eh[0].hex()), ('e_type', eh[1]), ('e_machine', eh[2]),
('e_version', eh[3]),
('e_entry', e_entry), ('e_phoff', e_phoff), ('e_shoff', e_shoff),
('e_flags', eh[7]),
('e_ehsize', eh[8]), ('e_phentsize', e_phentsize), ('e_phnum',
e_phnum),
('e_shentsize', e_shentsize), ('e_shnum', e_shnum), ('e_shstrndx',
e_shstrndx)]:
    print(' %-12s %s' % (n, v if isinstance(v, str) else hex(v)))

PT = {0: 'PT_NULL', 1: 'LOAD', 2: 'DYNAMIC', 3: 'INTERP', 4: 'NOTE',
6: 'PT_PHDR', 7: 'PT_TLS', 0x6474e550: 'GNU_EH_FRAME', 0x6474e551:
'GNU_STACK',
0x6474e552: 'GNU_RELRO', 0x6474e553: 'GNU_PROPERTY'}
FL = {0: '-', 1: 'X', 2: 'W', 3: 'WX', 4: 'R', 5: 'RX', 6: 'RW', 7: 'RWX'}

print('--- program headers (%d) ---' % e_phnum)
for i in range(e_phnum):
    p = struct.unpack_from('<IIQQQQQ', d, e_phoff + i * e_phentsize)
    kind = PT.get(p[0], 'type=%#x' % p[0])
    print('PH%d %-12s %-3s off=%#-8x vaddr=%#-9x filesz=%#-8x memsz=%#-8x
align=%#x' % (
        i, kind, FL.get(p[1], '?'), p[2], p[3], p[5], p[6], p[7]))

print('--- section headers (%d) ---' % e_shnum)
for i in range(e_shnum):
    s = struct.unpack_from('<IIQQQIIQQ', d, e_shoff + i * e_shentsize)
    print('SH%d' % i, s)
if e_shnum == 0:
    print(' (e_shoff=%#x e_shnum=0 -> 节头表被整表删除)' % e_shoff)

```

运行, 得到

```

len 10040
---

UPX      0x196a 0x199a 0x19bf 0x270b 0x2714      5
UPX0     -                                           0
UPX1     -                                           0
UPX!     0x270b 0x2714                                2
$$$      -                                           0
NRV      -                                           0
MPRESS   -                                           0
ASPack   -                                           0
FSG      -                                           0
.text    -                                           0
.data    -                                           0
.shstrtab -                                           0

```

```

--- ELF header ---
e_entry      0x405de0
e_phoff      0x40
e_shoff      0x0
e_phnum      0x5
e_shnum      0x0
e_shstrndx   0x0
--- program headers (5) ---
PH0  LOAD      RW  off=0x0      vaddr=0x400000  filesz=0x1000  memsz=0x4050
      align=0x1000
PH1  LOAD      RX  off=0x0      vaddr=0x405000  filesz=0x19e1  memsz=0x19e1
      align=0x1000
PH2  NOTE      R   off=0x158     vaddr=0x400158  filesz=0x30    memsz=0x30
      align=0x8
PH3  NOTE      R   off=0x188     vaddr=0x400188  filesz=0x44    memsz=0x44
      align=0x4
PH4  GNU_STACK RW  off=0x0      vaddr=0x0       filesz=0x0     memsz=0x0
      align=0x10
--- section headers (0) ---

```

看看这里有哪些信息。

- e_shoff=0x0 e_shnum=0 -> 节头表被整表删除
- 两个 PT_LOAD 的 p_offset 都被清零 (正常 ELF 第二个 LOAD 不可能和第一个同为 0) ;
- 节头表被整表删除 (e_shoff = e_shnum = 0) , hex 里搜不到 .text / .data 这类节名字符串;
- PHT 里只剩 5 项, PT_INTERP / PT_DYNAMIC / PT_GNU_RELRO / PT_GNU_EH_FRAME 全没了, PT_GNU_STACK 的 off/vaddr/filesz 也被清零;
- 尾部元数据自相矛盾 (见 1.3) 。

尽管知道魔改, 我们还是upx解一下试试

```

sunlight_tr4vel3r@Sleepymas07 /m/d> upx -d chall9.3
          Ultimate Packer for executables
          Copyright (C) 1996 - 2026
UPX 5.2.0      Markus Oberhumer, Laszlo Molnar & John Reiser      Jun 8th 2026

      File size      Ratio      Format      Name
      -----
upx: chall9.3: CantUnpackException: l_info corrupted

Unpacked 1 file: 0 ok, 1 error.

```

好的, 我们看到是l_info被破坏。

这里直接改l_info字段也一样报错, 不如我们直接提取能用的内容。

```
sun11ght_tr4vel3r@Sleepymas07 /m/d [1]> readelf -n chall9.3 | grep -E
"IBT|SHSTK"
Properties: x86 feature: IBT, SHSTK
```

确认一下是CET，那么entry point address处一定有指令endbr64

并且

先扫一下Entry point address，写个python脚本

```
# scan.py
import re, sys
fn = sys.argv[1] if len(sys.argv) > 1 else 'chall9.3'
d = open(fn, 'rb').read()
pats = {
    'endbr64': b'\xf3\x0f\x1e\xfa',
    'prologue endbr64;push rbp;mov rbp,rspace':
b'\xf3\x0f\x1e\xfa\x55\x48\x89\xe5',
    'push rbp;mov rbp,rspace': b'\x55\x48\x89\xe5',
    'ELFmagic': b'\x7fELF',
}
for name, p in pats.items():
    offs = [m.start() for m in re.finditer(re.escape(p), d)]
    print('%s (%d):' % (name, len(offs)))
    print('    ', ' '.join('%#x' % o for o in offs))
```

```
sun11ght_tr4vel3r@Sleepymas07 /m/d> python scan.py chall9.3
endbr64 (24):
    0x440 0x470 0x480 0x490 0x4a0 0x4b0 0x4c0 0x4d0 0x4e0 0x4f0 0x500 0x510 0x540
0x5c0 0x5f0 0x5f6 0x63f 0x675 0x924 0xb54 0xde0 0xf4c 0xf70 0xf9c
prologue endbr64;push rbp;mov rbp,rspace (4):
    0x5f6 0x63f 0x675 0x924
push rbp;mov rbp,rspace (5):
    0x5cd 0x5fa 0x643 0x679 0x928
ELFmagic (1):
    0x0
```

通过查阅资料，带 CET 的 gcc 产物里，**每个函数开头都有** `f3 0f 1e fa`（endbr64），所以扫这个 4 字节模式就能把“哪几段是代码”直接画出来。

endbr64 分成**两簇**：0x440..0xb54 和 0xde0..0xf9c。前者是原程序（从 CRT 到 `_fini`），后者就是 1.4 里那段 UPX stub。也就是说 0x440..0xb61 **这 1825 字节是明文机器码**。

看看入口处的 UPX stub

```
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)),
    '..', 'capstone_pkg'))
import capstone

d = open('chall9.3', 'rb').read()
md = capstone.Cs(capstone.CS_ARCH_X86, capstone.CS_MODE_64)
```

```

start = int(sys.argv[1], 0)
length = int(sys.argv[2], 0) if len(sys.argv) > 2 else 256
base = int(sys.argv[3], 0) if len(sys.argv) > 3 else 0x400000

buf = d[start:start + length]
for ins in md.disasm(buf, base + start):
    print('%08x  %-20s  %s  %s' % (ins.address, ins.bytes.hex(), ins.mnemonic,
ins.op_str))

```

看看几个偏移量，基本能推出原始偏移量0x405000。下面是输出。

```

sun11ght_tr4vel3r@sleepymas07 /m/d> python dis.py 0xde0 0x260 0x405000
00405de0  f30f1efa          endbr64
00405de4  59                pop rcx
00405de5  54                push rsp
00405de6  5f                pop rdi
00405de7  51                push rcx
00405de8  52                push rdx
00405de9  4c8d3d5c010000    lea r15, [rip + 0x15c]
00405df0  31c0              xor eax, eax
00405df2  48af              scasd rax, qword ptr [rdi]
00405df4  75fc              jne 0x405df2
00405df6  48af              scasd rax, qword ptr [rdi]
00405df8  75fc              jne 0x405df6
00405dfa  57                push rdi
00405dfb  ba00100000        mov edx, 0x1000
00405e00  5e                pop rsi
00405e01  48ad              lodsq rax, qword ptr [rsi]
00405e03  85c0              test eax, eax
00405e05  7408              je 0x405e0f
00405e07  83f806            cmp eax, 6
00405e0a  48ad              lodsq rax, qword ptr [rsi]
00405e0c  75f3              jne 0x405e01
00405e0e  92                xchg edx, eax
00405e0f  58                pop rax
00405e10  48f7da            neg rdx
00405e13  52                push rdx
00405e14  50                push rax
00405e15  6875705800        push 0x587075
00405e1a  54                push rsp
00405e1b  5f                pop rdi
00405e1c  6a10              push 0x10
00405e1e  5e                pop rsi
00405e1f  b83f010000        mov eax, 0x13f
00405e24  0f05              syscall
00405e26  85c0              test eax, eax
00405e28  791d              jns 0x405e47
00405e2a  85f6              test esi, esi
00405e2c  7404              je 0x405e32
00405e2e  31f6              xor esi, esi
00405e30  ebed              jmp 0x405e1f
00405e32  488d3526010000    lea rsi, [rip + 0x126]
00405e39  ad                lodsd eax, dword ptr [rsi]
00405e3a  92                xchg edx, eax
00405e3b  ad                lodsd eax, dword ptr [rsi]
00405e3c  56                push rsi

```

00405e3d	96	xchg esi, eax
00405e3e	5f	pop rdi
00405e3f	6a02	push 2
00405e41	58	pop rax
00405e42	e80a010000	call 0x405f51
00405e47	4989c4	mov r12, rax
00405e4a	59	pop rcx
00405e4b	488d3d8affffff	lea rdi, [rip - 0x76]
00405e52	8b0f	mov ecx, dword ptr [rdi]
00405e54	4829cf	sub rdi, rcx
00405e57	488d3526010000	lea rsi, [rip + 0x126]
00405e5e	ad	lodsdx eax, dword ptr [rsi]
00405e5f	93	xchg ebx, eax
00405e60	4189dd	mov r13d, ebx
00405e63	ad	lodsdx eax, dword ptr [rsi]
00405e64	92	xchg edx, eax
00405e65	83c210	add edx, 0x10
00405e68	52	push rdx
00405e69	6aff	push -1
00405e6b	57	push rdi
00405e6c	4829d9	sub rcx, rbx
00405e6f	51	push rcx
00405e70	4801fb	add rbx, rdi
00405e73	53	push rbx
00405e74	54	push rsp
00405e75	5d	pop rbp
00405e76	4829d4	sub rsp, rdx
00405e79	4883e4c0	and rsp, 0xffffffffffffc0
00405e7d	54	push rsp
00405e7e	5f	pop rdi
00405e7f	ad	lodsdx eax, dword ptr [rsi]
00405e80	50	push rax
00405e81	ad	lodsdx eax, dword ptr [rsi]
00405e82	6683f802	cmp ax, 2
00405e86	7401	je 0x405e89
00405e88	f4	hlt
00405e89	58	pop rax
00405e8a	4801f0	add rax, rsi
00405e8d	50	push rax
00405e8e	55	push rbp
00405e8f	31db	xor ebx, ebx
00405e91	31c9	xor ecx, ecx
00405e93	488d15d6000000	lea rdx, [rip + 0xd6]
00405e9a	6aff	push -1
00405e9c	5d	pop rbp
00405e9d	fc	cld
00405e9e	a8a4	test al, 0xa4
00405ea0	ffd2	call rdx
00405ea2	72fb	jb 0x405e9f
00405ea4	8d4101	lea eax, [rcx + 1]
00405ea7	ffd2	call rdx
00405ea9	11c0	adc eax, eax
00405eab	ffd2	call rdx
00405ead	73f8	jae 0x405ea7
00405eaf	83e803	sub eax, 3
00405eb2	720c	jb 0x405ec0
00405eb4	c1e008	shl eax, 8
00405eb7	ac	lodsb al, byte ptr [rsi]

00405eb8	83f0ff	xor eax, 0xffffffff
00405ebb	742f	je 0x405eec
00405ebd	4863e8	movsxd rbp, eax
00405ec0	8d4101	lea eax, [rcx + 1]
00405ec3	ffd2	call rdx
00405ec5	11c9	adc ecx, ecx
00405ec7	ffd2	call rdx
00405ec9	11c9	adc ecx, ecx
00405ecb	750d	jne 0x405eda
00405ecd	89c1	mov ecx, eax
00405ecf	83c002	add eax, 2
00405ed2	ffd2	call rdx
00405ed4	11c9	adc ecx, ecx
00405ed6	ffd2	call rdx
00405ed8	73f8	jae 0x405ed2
00405eda	81fd00f3ffff	cmp ebp, 0xffffffff300
00405ee0	11c1	adc ecx, eax
00405ee2	56	push rsi
00405ee3	488d342f	lea rsi, [rdi + rbp]
00405ee7	f3a4	rep movsb byte ptr [rdi], byte ptr [rsi]
00405ee9	5e	pop rsi
00405eea	ebb4	jmp 0x405ea0
00405eec	5d	pop rbp
00405eed	59	pop rcx
00405eee	4839ce	cmp rsi, rcx
00405ef1	7401	je 0x405ef4
00405ef3	f4	hlt
00405ef4	488b4530	mov rax, qword ptr [rbp + 0x30]
00405ef8	48890424	mov qword ptr [rsp], rax
00405efc	4c897c2408	mov qword ptr [rsp + 8], r15
00405f01	488b5520	mov rdx, qword ptr [rbp + 0x20]
00405f05	4883ea10	sub rdx, 0x10
00405f09	54	push rsp
00405f0a	5e	pop rsi
00405f0b	4c89e7	mov rdi, r12
00405f0e	52	push rdx
00405f0f	6a01	push 1
00405f11	58	pop rax
00405f12	e83a000000	call 0x405f51
00405f17	4801c6	add rsi, rax
00405f1a	29c2	sub edx, eax
00405f1c	75f1	jne 0x405f0f
00405f1e	5e	pop rsi
00405f1f	55	push rbp
00405f20	5c	pop rsp
00405f21	4d31c9	xor r9, r9
00405f24	4d89e0	mov r8, r12
00405f27	6a01	push 1
00405f29	415a	pop r10
00405f2b	6a05	push 5
00405f2d	5a	pop rdx
00405f2e	29ff	sub edi, edi
00405f30	6a09	push 9
00405f32	58	pop rax
00405f33	e819000000	call 0x405f51
00405f38	50	push rax
00405f39	48894518	mov qword ptr [rbp + 0x18], rax
00405f3d	4150	push r8

```

00405f3f 5f          pop rdi
00405f40 6a03        push 3
00405f42 58          pop rax
00405f43 0f05        syscall
00405f45 58          pop rax
00405f46 4883c010    add rax, 0x10
00405f4a ffe0        jmp rax
00405f4c f30f1efa    endbr64
00405f50 c3          ret
00405f51 50          push rax
00405f52 0f05        syscall
00405f54 59          pop rcx
00405f55 483d00f0ffff cmp rax, -0x1000
00405f5b 7201        jb 0x405f5e
00405f5d f4          hlt
00405f5e c3          ret
00405f5f c00100      rol byte ptr [rcx], 0
00405f62 0002        add byte ptr [rdx], al
00405f64 004100      add byte ptr [rcx], al

```

接下来寻找.text明文，稍微借用ai的力量

```

# locate.py — 只调上面的函数，把三个数打出来
import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from solve import locate_text

data = open(sys.argv[1] if len(sys.argv) > 1 else 'chall9.3', 'rb').read()
delta, main_vaddr, main_off = locate_text(data)
start, end = 0x401000 - delta, 0x401721 - delta
print('delta      = %#x      (vaddr = file_off + delta)' % delta)
print('main        = vaddr %#x @ file %#x' % (main_vaddr, main_off))
print('明文 .text = file[%#x:%#x] (%d bytes) -> vaddr %#x..%#x'
      % (start, end, end - start, start + delta, end + delta))

```

运行

```

delta      = 0x400bc0      (vaddr = file_off + delta)
main        = vaddr 0x4014e4 @ file 0x924
明文 .text = file[0x440:0xb61] (1825 bytes) -> vaddr 0x401000..0x401721

```

地址都有了，我们去把明文抠出来

```

# extract_text.py
import sys, os
from pathlib import Path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from solve import locate_text

src = Path(sys.argv[1] if len(sys.argv) > 1 else '../chall9.3')
data = src.read_bytes()

delta, main_vaddr, main_off = locate_text(data)
start = 0x401000 - delta      # _init
end = 0x401721 - delta      # _fini 结束
out = Path(str(src) + '.text.bin')

```

```

out.write_bytes(data[start:end])
print('vaddr = file_off + %#x' % delta)
print('extract file[%#x:%#x] (%d bytes) vaddr 0x401000..0x401721' % (start,
end, end - start))

```

```

vaddr = file_off + 0x400bc0
extract file[0x440:0xb61] (1825 bytes) vaddr 0x401000..0x401721
saved -> chall9.3.text.bin

```

老办法!

IDA里 File → open as binary, Processor 选 x86-64, Base 填 0x401000, 就能像正常程序一样 F5

那么现在只要翻译main函数

```

int main(void)
{
    char buf[0x80];
    printf("Input flag: "); // 0x4010A0 = printf
    if (fgets(buf, 0x80, stdin) == NULL) { // 0x4010C0 = fgets, rdx =
        stdin@0x404040
        puts("Wrong!"); // 0x401080 = puts
        return 0;
    }
    buf[strcspn(buf, "\n")] = 0; // 0x4010B0 = strcspn
    if (strlen(buf) != 0x30) { // 0x401090 = strlen -> 必须
        48 字节
        puts("Wrong!");
        return 0;
    }
    u64 a = be64(buf+0x00), b = be64(buf+0x08), c = be64(buf+0x10),
        d = be64(buf+0x18), e = be64(buf+0x20), f = be64(buf+0x28); // 大端!
    printf(<提示串 @0x402050>);
    printf(<提示串 @0x4020B0>);
    transform(&a, &b, &c, &d, &e, &f); // 0x401235
    if (a == 0xA9081F365437449Eu11 && b == 0xFDD155089F5F7434u11 &&
        c == 0x1999AD56BB2AE726u11 && d == 0x3B8513ED7842A8B6u11 &&
        e == 0x4232E4969CC61BBFu11 && f == 0xCF103310C7E723D7u11)
        puts("Correct!");
    else
        puts("Wrong!");
    return 0;
}

```

再找到transform函数

```

void transform(u64 *pa, u64 *pb, u64 *pc, u64 *pd, u64 *pe, u64 *pf)
{
    for (int i = 0; i <= 0x7E9; i++) { // 2026 轮
        u64 A=*pa, B=*pb, C=*pc, D=*pd, E=*pe, F=*pf;

        /* 1) 六个字各自循环左移 */
    }
}

```

```

A = rol(A, 11); B = rol(B, 45); C = rol(C, 14);
D = rol(D, 14); E = rol(E, 51); F = rol(F, 41);

/* 2) 前四个字加常量 */
A += 0x0123456789ABCDEFu11;
B += 0xFEDCBA9876543210u11;
C += 0x2152411021524111u11;
D += 0xECEBECEBECEBECECu11;

/* 3) 四条 XOR 级联（顺序固定，后面要用前面刚算出来的值） */
E ^= F ^ A ^ B ^ C;
F ^= B ^ E ^ D ^ C;
C ^= A ^ B ^ D ^ E;
B ^= C ^ D ^ E ^ F;

/* 4) 置换 */
*pa = B; *pb = D; *pc = F; *pd = A; *pe = C; *pf = E;
}
}

```

把工作汇总给ai，得到最后的答案，ai还自己把解答过程整合了一下

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""MoeCTF chall9.3 -- python solve.py ../chall9.3"""
import re, sys
from pathlib import Path

M64 = (1 << 64) - 1

# ----- 1. 定位明文 .text 并求 delta -----
START_SIG =
bytes.fromhex('f30f1efa31ed4989d15e4889e24883e4f050544531c031c948c7c7')
MAIN_SIG = bytes.fromhex('f30f1efa554889e54881ecb0000000')

def locate_text(data: bytes):
    """返回 (delta, main_vaddr, main_off): vaddr = file_off + delta."""
    m = re.search(re.escape(START_SIG), data)
    if not m:
        raise SystemExit('找不到 _start 特征')
    main_vaddr = int.from_bytes(data[m.end():m.end() + 4], 'little')
    mm = re.search(re.escape(MAIN_SIG), data[m.end():])
    if not mm:
        raise SystemExit('找不到 main 特征')
    main_off = m.end() + mm.start()
    return main_vaddr - main_off, main_vaddr, main_off

# ----- 2. 轮函数（可逆） -----
ROT = (11, 45, 14, 14, 51, 41) # a..f 的 rol 位数
ADD = (0x0123456789ABCDEF, 0xFEDCBA9876543210,
        0x2152411021524111, 0xECEBECEBECEBECEC) # a,b,c,d 每轮加上的常量
ROUNDS = 0x7E9 + 1 # cmp dword[rbp-4],0x7e9 /
jle -> 2026 轮
TARGET = (0xA9081F365437449E, 0xFDD155089F5F7434, 0x1999AD56BB2AE726,
           0x3B8513ED7842A8B6, 0x4232E4969CC61BBF, 0xCF103310C7E723D7)

def rol(v, n):

```

```

n &= 63
return ((v << n) | (v >> (64 - n))) & M64 if n else v & M64

def ror(v, n):
    n &= 63
    return ((v >> n) | (v << (64 - n))) & M64 if n else v & M64

def transform(a, b, c, d, e, f):
    """0x401235 的等价实现 (forward) 。"""
    for _ in range(ROUNDS):
        a, b, c, d, e, f = (rol(a, 11), rol(b, 45), rol(c, 14),
                             rol(d, 14), rol(e, 51), rol(f, 41))
        a = (a + ADD[0]) & M64; b = (b + ADD[1]) & M64
        c = (c + ADD[2]) & M64; d = (d + ADD[3]) & M64
        # 四条 XOR (顺序不能换, 后面的用到前面刚算出的值)
        e ^= f ^ a ^ b ^ c
        f ^= b ^ e ^ d ^ c
        c ^= a ^ b ^ d ^ e
        b ^= c ^ d ^ e ^ f
        a, b, c, d, e, f = b, d, f, a, c, e          # 置换
    return a, b, c, d, e, f

def transform_inv(a, b, c, d, e, f):
    """transform 的逆。"""
    for _ in range(ROUNDS):
        a, b, c, d, e, f = d, a, e, b, f, c          # 逆置换
        b ^= c ^ d ^ e ^ f                            # 逆 XOR (把上面四条按相反顺序解
        出来)
        c ^= a ^ b ^ d ^ e
        f ^= b ^ e ^ d ^ c
        e ^= f ^ a ^ b ^ c
        a = (a - ADD[0]) & M64; b = (b - ADD[1]) & M64    # 逆加
        c = (c - ADD[2]) & M64; d = (d - ADD[3]) & M64
        a, b, c, d, e, f = (ror(a, 11), ror(b, 45), ror(c, 14),
                             ror(d, 14), ror(e, 51), ror(f, 41))    # 逆 rol
    return a, b, c, d, e, f

def main():
    data = Path(sys.argv[1] if len(sys.argv) > 1 else
'../chall9.3').read_bytes()
    delta, main_vaddr, main_off = locate_text(data)
    print('[+] 明文 .text 定位成功')
    print('    vaddr = file_off + %#x' % delta)
    print('    main vaddr=%#x file_off=%#x' % (main_vaddr, main_off))

    t = (1, 2, 3, 4, 5, 6)
    assert transform_inv(*transform(*t)) == t
    print('[+] 轮函数往返自检通过 (2026 轮)')

    words = transform_inv(*TARGET)
    flag = b''.join(w.to_bytes(8, 'big') for w in words)    # be64 读入 -> 大端还原
    assert transform(*words) == TARGET, 'forward 回代失败'
    assert len(flag) == 0x30, '长度不满足 strlen(buf)==0x30'
    print('[+] forward(flag) == 目标常量 校验通过')
    print('[+] len == 0x30 == 48 校验通过')
    print()
    print('    6 个 u64 :', ' '.join('%#018x' % w for w in words))
    print('    Flag      : %s' % flag.decode())

```

```
if __name__ == '__main__':
    main()
```

```
$ python c93/solve.py chall9.3
```

```
[+] 明文 .text 定位成功
```

```
    vaddr = file_off + 0x400bc0
```

```
    main vaddr=0x4014e4 file_off=0x924
```

```
[+] 轮函数往返自检通过（2026 轮）
```

```
[+] forward(flag) == 目标常量 校验通过
```

```
[+] len == 0x30 == 48 校验通过
```

```
    6 个 u64 : 0x6d6f656374667b4d 0x306431666933645f 0x5550585f436f6d62
0x316e65645f773154 0x685f4c6974746c33 0x5f4372797074307d
    Flag      : moectf{M0d1fi3d_UPX_Comb1ned_w1Th_Littl3_Crypt0}
```

flag: moectf{M0d1fi3d_UPX_Comb1ned_w1Th_Littl3_Crypt0}

```
sunlight_tr4vel3r:sunlight_traveler ~/Ultato Polosion ... in MoeCTF 2026 [07:08:04]
$ moectf{M0d1fi3d_UPX_Comb1ned_w1Th_Littl3_Crypt0}
[*] 正在提交: moectf{M0d1fi3d_UPX_Comb1ned_w1Th_Littl3_Crypt0}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Ultato Polosion ... in MoeCTF 2026 [07:18:57]
```

请fanchai喝茶

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
    char *v3; // rsi
    __m128i *v4; // rdi
    __int64 v5; // rcx
    unsigned int v6; // edx
    unsigned __int32 v7; // esi
    unsigned __int32 v8; // ebp
    unsigned __int32 v9; // ebx
    unsigned __int32 v10; // r11d
    unsigned __int32 v11; // r10d
    unsigned __int32 v12; // r9d
    unsigned __int32 v13; // r8d
    unsigned __int32 v14; // edi
    __m128i v15; // xmm0
    __m128i v16; // xmm0
    unsigned __int32 v18; // [rsp+0h] [rbp-E8h]
    unsigned __int32 v19; // [rsp+4h] [rbp-E4h]
    unsigned __int32 v20; // [rsp+8h] [rbp-E0h]
    unsigned __int32 v21; // [rsp+Ch] [rbp-DCh]
    unsigned __int32 v22; // [rsp+10h] [rbp-D8h]
    unsigned __int32 v23; // [rsp+14h] [rbp-D4h]
    unsigned __int32 v24; // [rsp+18h] [rbp-D0h]
    unsigned __int32 v25; // [rsp+1Ch] [rbp-CCh]
    __m128i v26; // [rsp+20h] [rbp-C8h] BYREF
    __m128i v27; // [rsp+30h] [rbp-B8h]
    __m128i v28; // [rsp+40h] [rbp-A8h]
    __m128i v29; // [rsp+50h] [rbp-98h]
    char s[136]; // [rsp+60h] [rbp-88h] BYREF
```

```

fwrite("Input flag: ", 1uLL, 0xCuLL, stdout);
fflush(stdout);
if ( !fgets(s, 66, stdin) )
    goto LABEL_9;
s[strcspn(s, "\r\n")] = 0;
if ( strlen(s) != 64 )
    goto LABEL_9;
v3 = s;
v4 = &v26;
v5 = 16LL;
v6 = 0;
while ( v5 )
{
    v4->m128i_i32[0] = *(_DWORD *)v3;
    v3 += 4;
    v4 = (__m128i *)((char *)v4 + 4);
    --v5;
}
v7 = v29.m128i_u32[3];
v8 = v28.m128i_i32[0];
v9 = v28.m128i_u32[1];
v10 = v28.m128i_u32[2];
v11 = v28.m128i_u32[3];
v25 = v26.m128i_i32[0];
v12 = v29.m128i_i32[0];
v13 = v29.m128i_u32[1];
v14 = v29.m128i_u32[2];
v20 = v26.m128i_u32[2];
v24 = v26.m128i_u32[1];
v19 = v26.m128i_u32[3];
v23 = v27.m128i_u32[1];
v18 = v27.m128i_i32[0];
v21 = v27.m128i_u32[3];
v22 = v27.m128i_u32[2];
do
{
    v6 += 1597463007;
    v25 += ((v6 ^ v24) + (v7 ^ dword_2020[(v6 >> 2) & 3])) ^ (((4 * v24) ^ (v7
>> 5)) + ((16 * v7) ^ (v24 >> 3)));
    v24 += (((16 * v25) ^ (v20 >> 3)) + ((4 * v20) ^ (v25 >> 5))) ^ ((v6 ^ v20)
+ (v25 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 1) & 3)]));
    v20 += (((16 * v24) ^ (v19 >> 3)) + ((4 * v19) ^ (v24 >> 5))) ^ ((v6 ^ v19)
+ (v24 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 2) & 3)]));
    v19 += (((16 * v20) ^ (v18 >> 3)) + ((4 * v18) ^ (v20 >> 5))) ^ ((v6 ^ v18)
+ (v20 ^
dword_2020[~((unsigned __int8)(v6 >> 2) & 3)]));
    v18 += (((16 * v19) ^ (v23 >> 3)) + ((4 * v23) ^ (v19 >> 5))) ^ ((v6 ^ v23)
+ (v19 ^ dword_2020[(v6 >> 2) & 3]));
    v23 += (((16 * v18) ^ (v22 >> 3)) + ((4 * v22) ^ (v18 >> 5))) ^ ((v6 ^ v22)
+ (v18 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 5) & 3)]));
    v22 += (((16 * v23) ^ (v21 >> 3)) + ((4 * v21) ^ (v23 >> 5))) ^ ((v6 ^ v21)
+ (v23 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 6) & 3)]));
    v21 += (((16 * v22) ^ (v8 >> 3)) + ((4 * v8) ^ (v22 >> 5))) ^ ((v8 ^ v6)

```

```

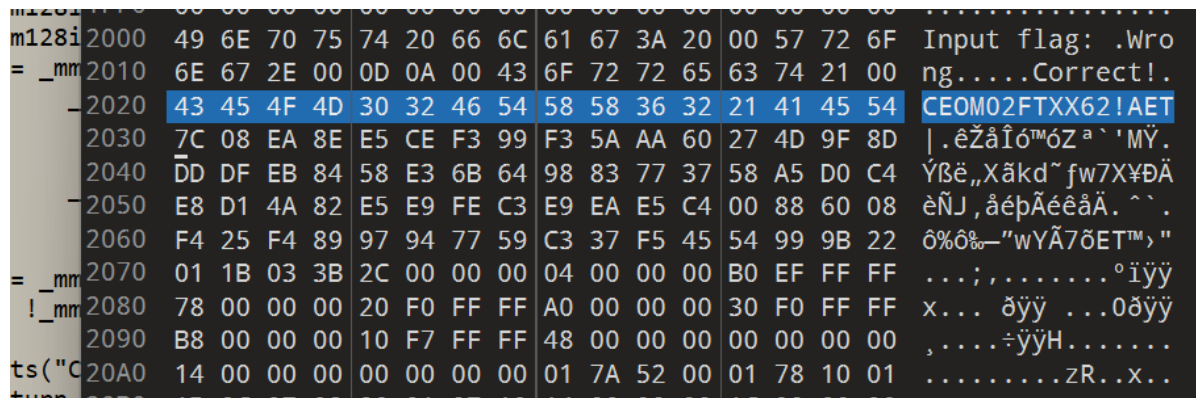
+ (v22 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 7) & 3]));
v8 += (((16 * v21) ^ (v9 >> 3)) + ((4 * v9) ^ (v21 >> 5))) ^ ((v9 ^ v6) +
(v21 ^ dword_2020[(v6 >> 2) & 3]));
v9 += (((16 * v8) ^ (v10 >> 3)) + ((4 * v10) ^ (v8 >> 5))) ^ ((v10 ^ v6)
+ (v8 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 9) & 3]));
v10 += (((16 * v9) ^ (v11 >> 3)) + ((4 * v11) ^ (v9 >> 5))) ^ ((v11 ^ v6)
+ (v9 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xA) & 3]));
v11 += (((16 * v10) ^ (v12 >> 3)) + ((4 * v12) ^ (v10 >> 5))) ^ ((v12 ^ v6)
+ (v10 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xB) & 3]));
v12 += (((16 * v11) ^ (v13 >> 3)) + ((4 * v13) ^ (v11 >> 5))) ^ ((v13 ^ v6)
+ (v11 ^ dword_2020[(v6 >> 2) & 3]));
v13 += (((v12 >> 5) ^ (4 * v14)) + ((v14 >> 3) ^ (16 * v12))) ^ ((v14 ^ v6)
+ (v12 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xD) & 3]));
v14 += (((v13 >> 5) ^ (4 * v7)) + ((v7 >> 3) ^ (16 * v13))) ^ ((v7 ^ v6)
+ (v13 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xE) & 3]));
v7 += (((16 * v14) ^ (v25 >> 3)) + ((4 * v25) ^ (v14 >> 5))) ^ ((v6 ^ v25)
+ (v14 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xF) & 3]));
}
while ( v6 != 1492265175 );
v28.m128i_i64[0] = __PAIR64__(v9, v8);
v26.m128i_i64[0] = __PAIR64__(v24, v25);
v28.m128i_i64[1] = __PAIR64__(v11, v10);
v26.m128i_i64[1] = __PAIR64__(v19, v20);
v29.m128i_i64[0] = __PAIR64__(v13, v12);
v27.m128i_i64[0] = __PAIR64__(v23, v18);
v29.m128i_i64[1] = __PAIR64__(v7, v14);
v27.m128i_i64[1] = __PAIR64__(v21, v22);
v15 = _mm_or_si128(
    _mm_or_si128(
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2030), v27),
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2040), v26)),
    _mm_or_si128(
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2050), v29),
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2060),
v28)));
v16 = _mm_or_si128(v15, _mm_srli_si128(v15, 8));
if ( !_mm_cvtsi128_si32(_mm_or_si128(v16, _mm_srli_si128(v16, 4))) )
{
    puts("Correct!");
    return 0LL;
}
else
{
    LABEL_9:
    puts("Wrong.");
    return 1LL;
}
}

```

tea加密我们之前见识过了，但是这次我们尝试自己写解密脚本。

现在我们需要xmmword_2030-2060, 以及dword_2020

直接用010Editor打开



ok了, 那么dword_2020是43 45 4F 4D 30 32 46 54 58 58 36 32 21 41 45 54 (密钥)

xmmword_2030是7C 08 EA 8E E5 CE F3 99 F3 5A AA 60 27 4D 9F 8D (密文)

xmmword_2040是DD DF EB 84 58 E3 6B 64 98 83 77 37 58 A5 D0 C4 (密文)

xmmword_2050是E8 D1 4A 82 E5 E9 FE C3 E9 EA E5 C4 00 88 60 08 (密文)

xmmword_2060是F4 25 F4 89 97 94 77 59 C3 37 F5 45 54 99 9B 22 (密文)

(在ida中需手算, 我们用010Editor省去计算)

那么我们写脚本吧! 参照维基百科上tea解密的标准写法和先去tea解密的写法。

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define DELTA 0x5F3759DF

#define MX (((z>>5^y<<2) + (y>>3^z<<4)) ^ ((sum^y) + (key[(p&3)^e] ^ z)))

void btea_decrypt(uint32_t *v, int n, uint32_t const key[4]) {
    uint32_t y, z, sum;
    unsigned p, rounds, e;
    rounds = 6 + 52 / n;
    sum = rounds * DELTA;
    y = v[0];
    do {
        e = (sum >> 2) & 3;
        for (p = n - 1; p > 0; p--) {
            z = v[p - 1];
            y = v[p] -= MX;
        }
        z = v[n - 1];
        y = v[0] -= MX;
        sum -= DELTA;
    } while (--rounds);
}

int main() {
```

```

uint32_t key[4] = {
    0x4D4F4543,
    0x54463230,
    0x32365858,
    0x54454121
};

uint32_t v[16] = {
    // xmmword_2040
    0x84EBDFDD, 0x646BE358, 0x37778398, 0xc4D0A558,
    // xmmword_2030
    0x8EEA087C, 0x99F3CEE5, 0x60AA5AF3, 0x8D9F4D27,
    // xmmword_2060
    0x89F425F4, 0x59779497, 0x45F537C3, 0x229B9954,
    // xmmword_2050
    0x824AD1E8, 0xC3FEE9E5, 0xC4E5EAE9, 0x08608800
};

int n = 16;
btea_decrypt(v, n, key);

char flag[65];
for (int i = 0; i < 16; i++) {
    flag[i * 4 + 0] = (v[i] >> 0) & 0xFF;
    flag[i * 4 + 1] = (v[i] >> 8) & 0xFF;
    flag[i * 4 + 2] = (v[i] >> 16) & 0xFF;
    flag[i * 4 + 3] = (v[i] >> 24) & 0xFF;
}
flag[64] = '\0';

printf("Flag: %s\n", flag);
return 0;
}

```

那么运行，结束

flag:

moectf{Tre4t_f4ncha1_W1tH_xxtea_Wh4t_A_g00d_Id3a_HahaHAh4hAH4ha}

```

sunlight_tr4vel3r:sunlight_traveler ~/请 fanchai 喝茶 — in MoeCTF 2026 [21:52:09]
$ moectf{Tre4t_f4ncha1_W1tH_xxtea_Wh4t_A_g00d_Id3a_HahaHAh4hAH4ha}
[*] 正在提交: moectf{Tre4t_f4ncha1_W1tH_xxtea_Wh4t_A_g00d_Id3a_HahaHAh4hAH4ha}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/请 fanchai 喝茶 — in MoeCTF 2026 [21:55:56]
$

```

广晨风语录

用jadx打开，先看看源码。

```

package com.moectf.gcfquotes;

import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

/* JADX INFO: loaded from: classes.dex */
public class MainActivity extends Activity {
    private EditText input;
    private boolean nativeLoaded;
    private TextView output;

    private native String nativeQuote(String str);

    @Override // android.app.Activity
    protected void onCreate(Bundle bundle) {
        super.onCreate(bundle);
        setContentView(R.layout.activity_main);
        this.input = (EditText) findViewById(R.id.input);
        this.output = (TextView) findViewById(R.id.output);
        ((Button) findViewById(R.id.submit)).setOnClickListener(new
view.OnClickListener() { // from class: com.moectf.gcfquotes.MainActivity.1
            @Override // android.view.View.OnClickListener
            public void onClick(View view) {
                MainActivity.this.submitInput();
            }
        });
    }

    /* JADX INFO: Access modifiers changed from: private */
    public void submitInput() {
        if (!this.nativeLoaded) {
            try {
                System.loadLibrary("gcquotes");
                this.nativeLoaded = true;
            } catch (Throwable th) {
                this.output.setText("超市就是大润发");
                return;
            }
        }
        try {
            String strNativeQuote = nativeQuote(this.input.getText().toString());
            if (strNativeQuote != null && strNativeQuote.length() != 0) {
                this.output.setText(strNativeQuote);
                return;
            }
            this.output.setText("电车就是问界");
        } catch (Throwable th2) {
            this.output.setText("手机就是遥遥领先");
        }
    }
}

```

眼尖的已经看到了：System.loadLibrary("gcquotes");

那么我们需要libgcquotes.so，最好是v8a架构。

右键导出即可。

用ida打开.so试试，发现挺复杂的。

别急，我们看看与flag有关系的是哪些。

首先发现一个gcf::InputSession::capture(char const*)

```
__int64 __fastcall gcf::InputSession::capture(gcf::InputSession *this, gcf *a2)
{
    const char *v4; // x1
    unsigned __int64 v5; // x0
    unsigned __int64 v6; // x3
    const unsigned __int8 *v7; // x21
    __int64 result; // x0

    gcf::InputSession::reset(this);
    v5 = gcf::local_strlen(a2, v4);
    if ( v5 >= 0x80 )
        v7 = (const unsigned __int8 *)&qword_80;
    else
        v7 = (const unsigned __int8 *)v5;
    result = gcf::local_copy(this, (unsigned __int8 *)a2, v7, v6);
    *((_QWORD *)this + 16) = v7;
    return result;
}
```

在这些函数上打开文本视图，在附近可以发现其他的函数，整理一下

地址	符号
0x8B7C	Java_com_moectf_gcfquotes_MainActivity_nativeQuote
0x86B0	gcf::QuoteEngine::process(char const*)
0x8334	gcf::InputSession::capture(char const*)
0x8388	gcf::InputSession::prepare()
0x8400	gcf::PayloadProcessor::transform(unsigned char*, unsigned long)
0x849C	gcf::Verifier::verify(gcf::QuoteRequest*)
0x8478	gcf::Verifier::compare(unsigned char const*, unsigned char const*, unsigned long)
0x8224	gcf::constant_time_equal(unsigned char const*, unsigned char const*, unsigned long)
0x8488	gcf::Verifier::expectedSize() → 常量 0x1b
0x8A08	gcf::nativeBuildNumber() → 0x13527CA

看到gcf::InputSession::prepare()的文本视图：

```

.text:0000000000008388 STP X29, X30, [SP, #-0x20+var_s0]!
.text:000000000000838C STP X20, X19, [SP, #var_s10]
.text:0000000000008390 MOV X29, SP
.text:0000000000008394 LDR X20, [X0, #0x80]
.text:0000000000008398 CBZ X20, loc_83E0
.text:000000000000839C MOV X1, X20; unsigned __int8 *
.text:00000000000083A0 MOV X19, X0
.text:00000000000083A4 BL .ZN3gcf16rolling_checksumEPKhm; gcf::rolling_checksum(uchar const*,ulong)
.text:00000000000083A8 MOV W8, #0x4659
.text:00000000000083AC CMP X20, #1
.text:00000000000083B0 MOVK W8, #0x4751, LSL#16
.text:00000000000083B4 EOR W8, W0, W8
.text:00000000000083B8 ADD W8, W8, W20
.text:00000000000083BC STP W0, W8, [X19, #0x88]
.text:00000000000083C0 NOP
.text:00000000000083C4 CMP W8, #0
.text:00000000000083C8 MOV X8, X19
.text:00000000000083CC LDRB W9, [X8]
.text:00000000000083D0 EOR W9, W9, #0xAAAAAAAA
.text:00000000000083D4 STRB W9, [X8]
.text:00000000000083D8 MOV W8, #1
.text:00000000000083DC STR W8, [X19, #0x90]
.text:00000000000083E0
.text:00000000000083E0 loc_83E0 ; CODE XREF: gcf::InputSession::prepare(void)+10↑j
.text:00000000000083E0 LDP X20, X19, [SP, #var_s10]
.text:00000000000083E4 LDP X29, X30, [SP+var_s0], #0x20
.text:00000000000083E8 RET
.text:00000000000083E8 ; } // starts at 8388
.text:00000000000083E8 ; End of function gcf::InputSession::prepare(void)

```

83D0处: eor w9, w9, #0xAAAAAAAA` 看起来像4字节异或但w9来自ldrb (只有低8位有效), 紧接着是strb, 所以实际效果就是session[0] ^= 0xAA, 其余 26 字节原样不动。

Verifier::verify的文本视图:

```

.text:000000000000849C
.text:000000000000849C ; ===== SUBROUTINE =====
.text:000000000000849C
.text:000000000000849C ; gcf::Verifier::verify(gcf::QuoteRequest *)
.text:000000000000849C EXPORT _ZN3gcf8Verifier6verifyEPNS_12QuoteRequestE
.text:000000000000849C _ZN3gcf8Verifier6verifyEPNS_12QuoteRequestE
.text:000000000000849C ; CODE XREF: gcf::Verifier::verify(gcf::QuoteRequest *)+C1j
.text:000000000000849C ; DATA XREF: LOAD:0000000000005A8To ...
.text:000000000000849C
.text:000000000000849C var_20 = -0x20
.text:000000000000849C var_18 = -0x18
.text:000000000000849C var_10 = -0x10
.text:000000000000849C
.text:000000000000849C ; _unwind {
.text:000000000000849C CBZ X1, loc_84F4
.text:00000000000084A0 LDR W8, [X1, #0x90]
.text:00000000000084A4 CBZ W8, loc_84F4
.text:00000000000084A8 LDR X8, [X1, #0x80]
.text:00000000000084AC CMP X8, #0x1B
.text:00000000000084B0 B.NE loc_84FC
.text:00000000000084B4 STP X29, X30, [SP, #var_20]!
.text:00000000000084B8 STR X19, [SP, #0x20+var_10]
.text:00000000000084BC MOV X29, SP
.text:00000000000084C0 W2, #0x1B; unsigned __int64
.text:00000000000084C4 MOV X19, X1
.text:00000000000084C8 BL .ZNK3gcf16PayloadProcessor9transformEPPhm; gcf::PayloadProcessor::transform(uchar *,ulong)
.text:00000000000084CC ADRL X2, unk_3020; unsigned __int8 *
.text:00000000000084D0 MOV X1, X19; gcf *
.text:00000000000084D4 MOV W3, #0x18+3; unsigned __int8 *
.text:00000000000084D8 BL .ZNK3gcf8Verifier7compareEPKhS2_m; gcf::Verifier::compare(uchar const*,uchar const*,ulong)
.text:00000000000084DC MOV X1, X19
.text:00000000000084E0 LDR X19, [SP, #0x20+var_10]
.text:00000000000084E4 LDP X29, X30, [SP+0x20+var_20], #0x20
.text:00000000000084E8 STR W0, [X1, #0x94]
.text:00000000000084F0 RFT

```

compare→constant_time_equal: 逐字节异或后按位或, 全0才返回1。

所以flag长度为27字节

gcf::PayloadProcessor::transform(unsigned char*, unsigned long)的文本视图:

```

.text:0000000000008400
.text:0000000000008400 ; ===== SUBROUTINE =====
.text:0000000000008400
.text:0000000000008400 ; __int64 __fastcall gcf::PayloadProcessor::transform(gcf::PayloadProcessor * _hidden this, unsigned __int8 *, unsigned __int
.text:0000000000008400 EXPORT _ZNK3gcf16PayloadProcessor9transformEPPhm
.text:0000000000008400 _ZNK3gcf16PayloadProcessor9transformEPPhm
.text:0000000000008400 ; CODE XREF: gcf::PayloadProcessor::transform(uchar *,ulong)+C1j
.text:0000000000008400 ; DATA XREF: LOAD:0000000000000D10To ...
.text:0000000000008400 ; _unwind {
.text:0000000000008400 CBZ X1, locret_8430
.text:0000000000008404 SUBS X2, X2, #1
.text:0000000000008408 B.LS locret_8430
.text:000000000000840C LDRB W8, [X1], #1
.text:0000000000008410 MOV W10, #0x67; 'g'
.text:0000000000008414
.text:0000000000008414 loc_8414 ; CODE XREF: gcf::PayloadProcessor::transform(uchar *,ulong)+2C1j
.text:0000000000008414 LDRB W11, [X1]
.text:0000000000008418 ADD W8, W8, #0x28; '('
.text:000000000000841C EOR W8, W8, W10
.text:0000000000008420 EOR W8, W11, W8
.text:0000000000008424 STRB W8, [X1], #1
.text:0000000000008428 SUBS X2, X2, #1
.text:000000000000842C B.NE loc_8414
.text:0000000000008430
.text:0000000000008430 locret_8430 ; CODE XREF: gcf::PayloadProcessor::transform(uchar *,ulong)↑j
.text:0000000000008430 ; gcf::PayloadProcessor::transform(uchar *,ulong)+81j
.text:0000000000008430 RET
.text:0000000000008430 ; } // starts at 8400
.text:0000000000008430 ; End of function gcf::PayloadProcessor::transform(uchar *,ulong)
.text:0000000000008430
0004400 0000000000008400: gcf::PayloadProcessor::transform(uchar *,ulong)

```

把循环展开：

```
w8 = buf[0]
x1 = buf+1
loop:
    w10 = buf[1]
    w8 = ((w8 + 0x28) ^ 0x67) ^ w10
    buf[1] = w8
    x1++
```

整理一下：buf[i] = buf[i] ^ (((buf[i-1] + 0x28) ^ 0x67) & 0xff), i = 1 .. n-1

在0x3020处有一张期望值表

```
.rodata:0000000000003020 unk_3020 DCB 0xC7 ; DATA XREF: gcf::Verifier::verify(gcf::QuoteRequest
.rodata:0000000000003021 DCB 0xE7
.rodata:0000000000003022 DCB 0xD
.rodata:0000000000003023 DCB 0x31 ; 1
.rodata:0000000000003024 DCB 0x4A ; J
.rodata:0000000000003025 DCB 0x73 ; s
.rodata:0000000000003026 DCB 0x87
.rodata:0000000000003027 DCB 0x8D
.rodata:0000000000003028 DCB 0xB3
.rodata:0000000000003029 DCB 0x89
.rodata:000000000000302A DCB 0xAF
.rodata:000000000000302B DCB 0xEF
.rodata:000000000000302C DCB 0x44 ; D
.rodata:000000000000302D DCB 0x65 ; e
.rodata:000000000000302E DCB 0x8E
.rodata:000000000000302F DCB 0xA3
.rodata:0000000000003030 DCB 0xC3
.rodata:0000000000003031 DCB 0xE5
.rodata:0000000000003032 DCB 0x2E ; .
.rodata:0000000000003033 DCB 0x6E ; n
.rodata:0000000000003034 DCB 0xBF
.rodata:0000000000003035 DCB 0xE1
.rodata:0000000000003036 DCB 0x1A
.rodata:0000000000003037 DCB 0x4C ; L
.rodata:0000000000003038 DCB 0x65 ; e
.rodata:0000000000003039 DCB 0xD9
.rodata:000000000000303A DCB 0x1B
```

那么就可以写个python脚本打印flag

```
T = bytes.fromhex('c7 e7 0d 31 4a 73 87 8d b3 89 af ef 44 65 8e a3 c3 e5 2e 6e
bf e1 1a 4c 65 d9 1b')

def inv_transform(buf):
    out = bytearray(buf)
    for i in range(1, len(out)):
        out[i] = out[i] ^ (((buf[i-1] + 0x28) ^ 0x67) & 0xff)
    return bytes(out)

raw = inv_transform(T)
flag = bytes([raw[0] ^ 0xAA]) + raw[1:]
print(flag)
```

运行，结束。

flag: moectf{Ea5y_4ndroid_Nativ3}

持续更新ing...

